

CS162

Operating Systems and Systems Programming

Lecture 19

File Systems

Professor Ion Stoica & Matei Zaharia

<https://cs162.org/>

Recall: Storage Devices

Magnetic disks (HDDs)

- Block-level random access interface
- Slow for random access due to mechanical seek + rotation delays (latency can be 5-10ms)
- Faster for sequential access (peak throughput 50-250 MB/s)

Flash memory (SSDs)

- Same block-level access interface
- Higher cost/byte (2-20x disk)
- No moving parts, so fast (latency $< 0.1\text{ms}$, throughput 3+ GB/s)
- Wear out after some number of writes!

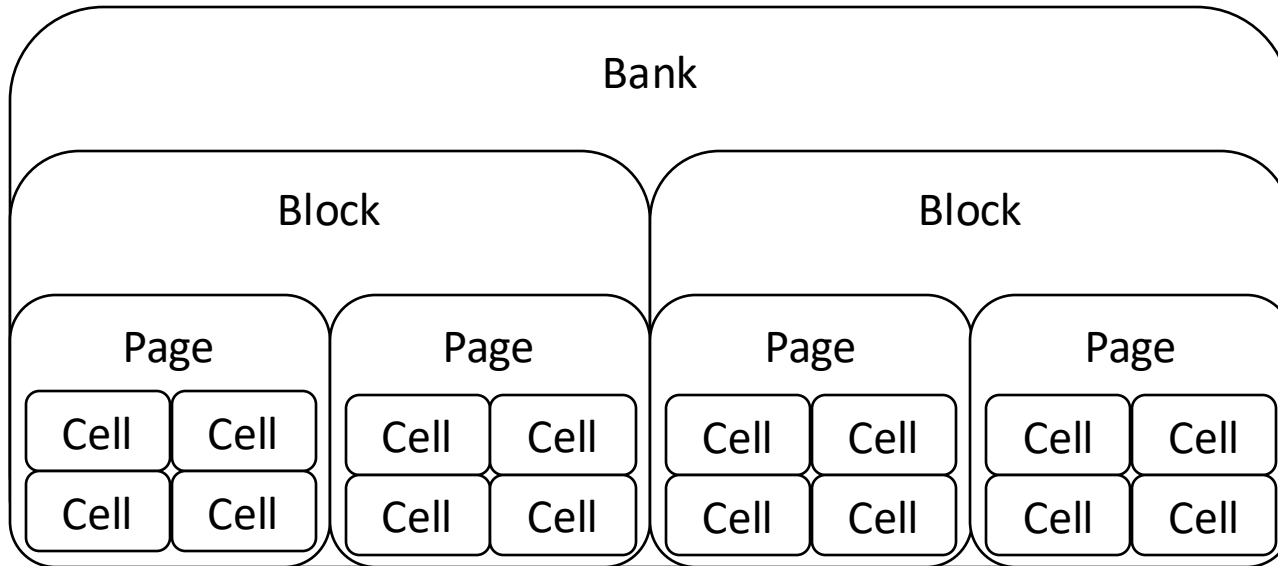
Recall: Banks, Blocks, and Cells in SSDs

Flash chips organized in **banks**

Blocks 128 KB/256KB

Pages Few KB

Cells 1 to 4 bits



Low-level Flash Operations

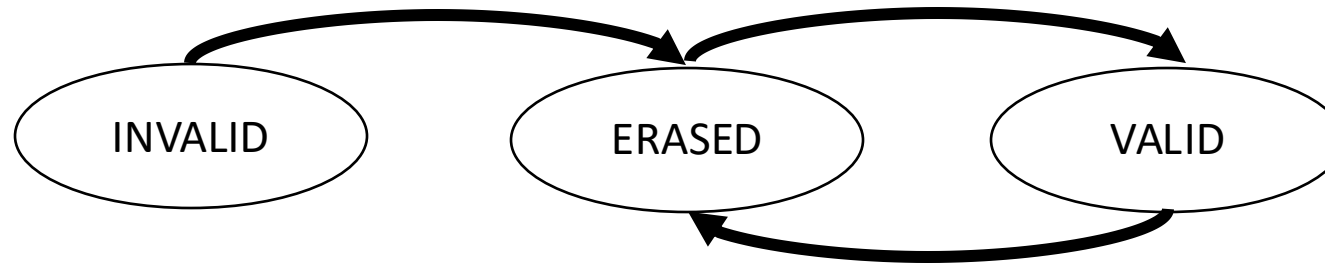
How do you **read**?

- Chip supports reading *pages*
- 10s of microseconds, independently of the previously read page

What about **writing**?

- Must first *erase the block*
 - » Expensive (milliseconds)
- Once block has been erased, can then *program a page*
 - » Change 1s to 0s within a page.
 - » 100s of microseconds.
- Blocks can only be erased a limited number of times!

Low-level Flash Operations



		<i>i i i i</i>	<i>Initial: pages in block are invalid (i)</i>
Erase()	→	<i>E E E E</i>	<i>State of pages in block set to erased (E)</i>
Program(0)	→	<i>V E E E</i>	<i>Program page 0; state set to valid (V)</i>
Program(0)	→	error	<i>Cannot re-program page after programming</i>
Program(1)	→	<i>V V E E</i>	<i>Program page 1</i>
Erase()	→	<i>E E E E</i>	<i>Contents erased; all pages programmable</i>

Making Writes More Efficient

Why not just erase and rewrite an entire 256KB block when we write to a page?

- Erasure is very slow (milliseconds)
- Blocks wear out, can only be rewritten about 10K times
- Heavily used blocks will wear out faster

Solution: Flash Translation Layer (FTL)

- **Add a layer of indirection:** map virtual block IDs (which the OS sees) to physical flash pages using a lookup table (usually in RAM)
 - » Reduce writes with copy-on-write: when a page is updated, move it to another flash block, and keep the old copy; then garbage-collect over time
 - » Balance wear-out across the chip

FTL Example

Initial
State

Mapping Table:			
Block 0		Block 1	
E	E	E	E

Write(a1)

Mapping Table: a0->0,0/a1->1,0			
Block 0		Block 1	
a0	a1	a1	
V	V	V	E

Write(a0)

Mapping Table a0->0,0			
Block 0		Block 1	
a0			
V	E	E	E

Write(a0)

Mapping Table: a0->1,1/a1->1,0			
Block 0		Block 1	
a0	a1	a1	a0
V	V	V	V

Write(a1)

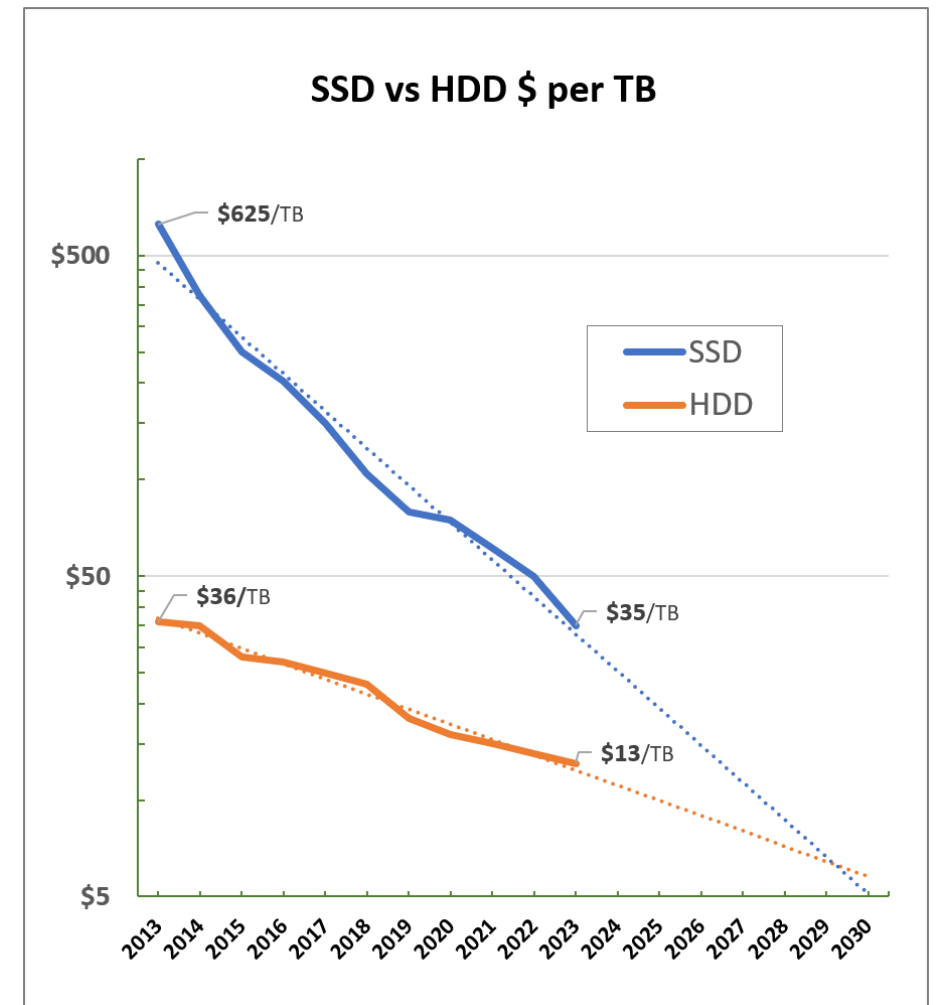
Mapping Table: a0->0,0/a1->0,1			
Block 0		Block 1	
a0	a1		
V	V	E	E

Garbage
Collect

Mapping Table: a0->1,1/a1->1,0			
Block 0		Block 1	
		a1	a0
E	E	V	V

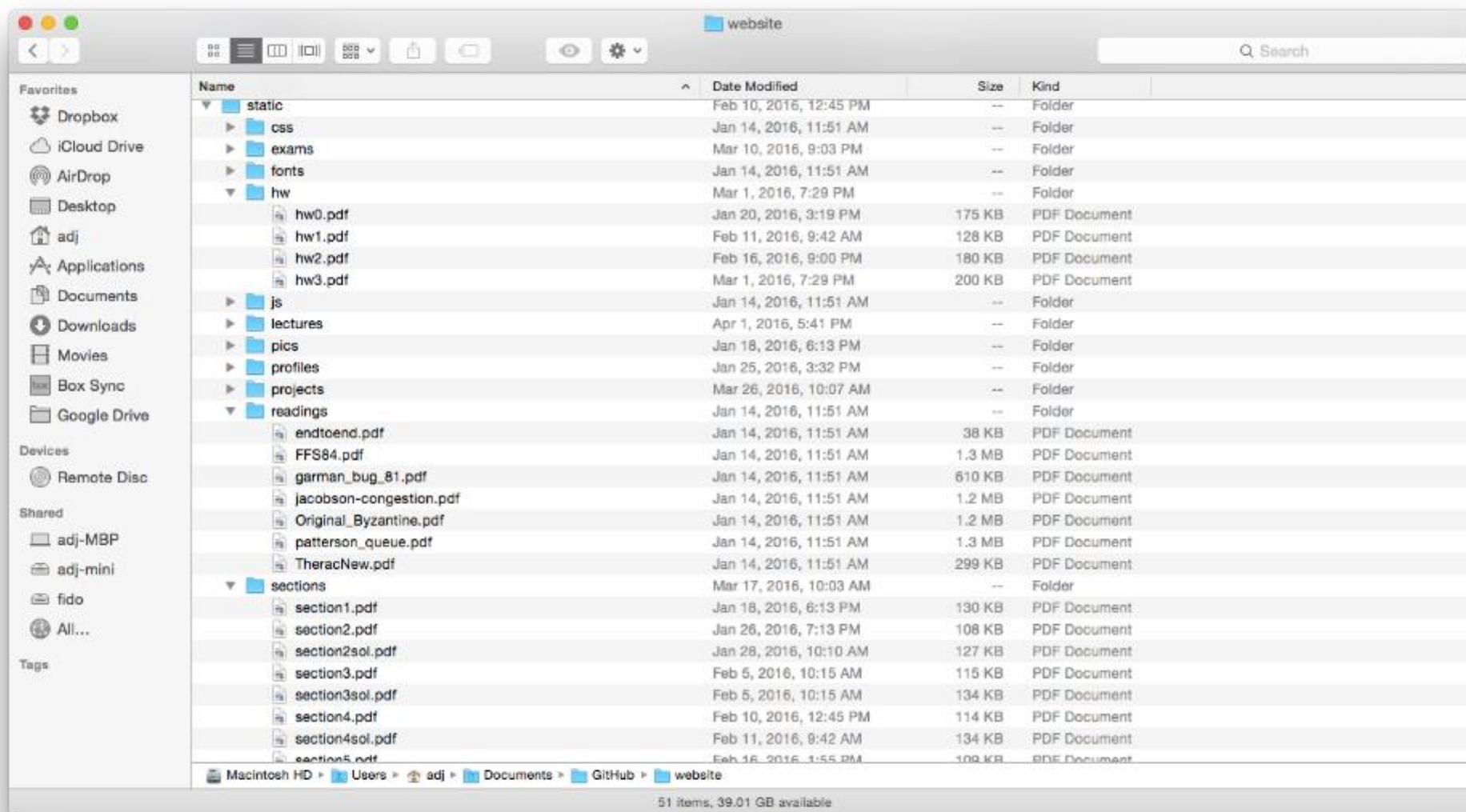
Summary: HDDs vs SSDs

HDD	SDD
Require seek + rotation	No moving parts
Not parallel (one head)	Parallel
Random reads take 10s of ms	Random reads take 10s of μ s
Peak tput 250 MB/s	Peak tput 3+ GB/s
Slow (Mechanical)	Limited write cycles
Cheap/large storage	Expensive/smaller

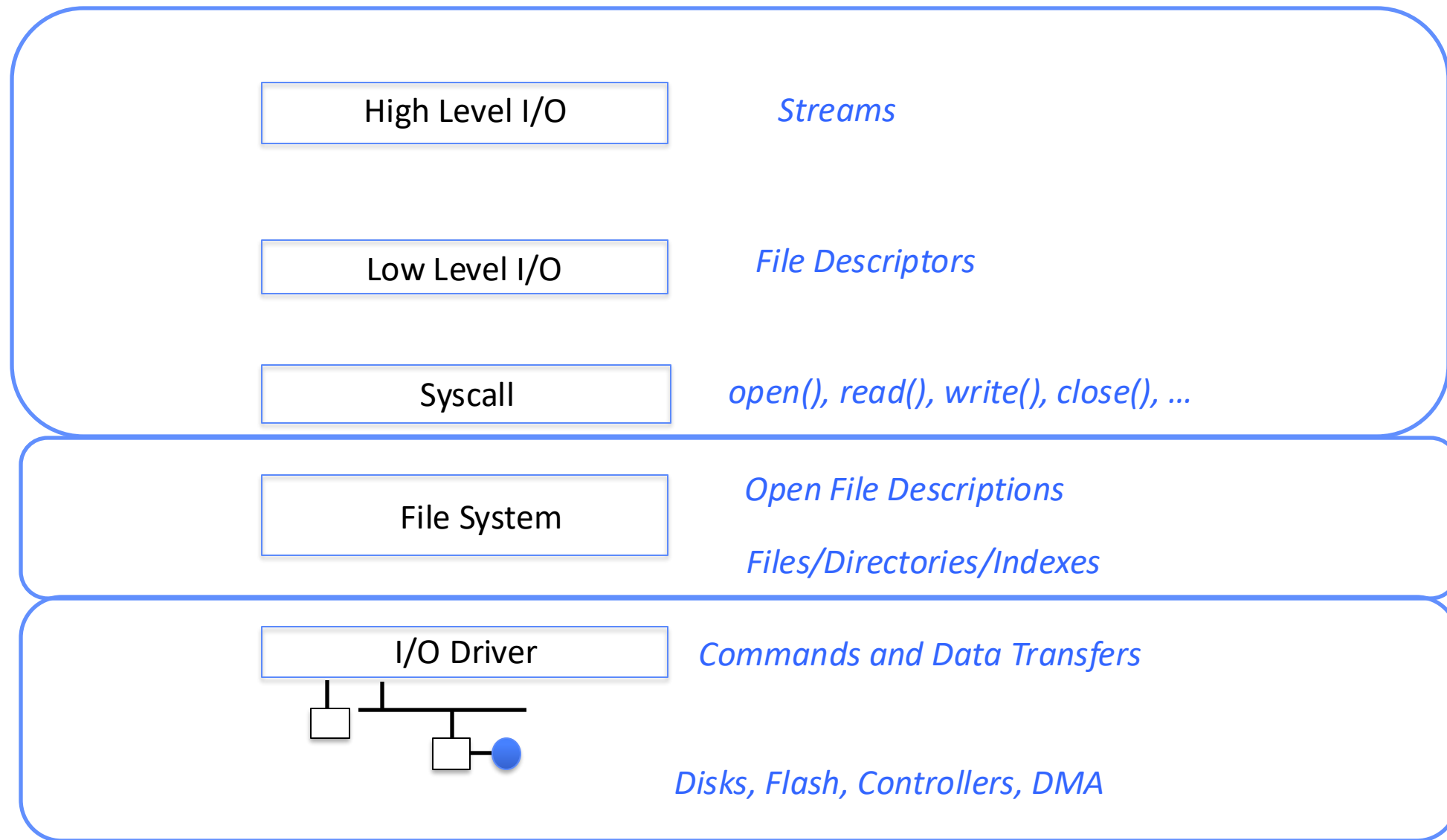


SSD prices falling faster than HDD!

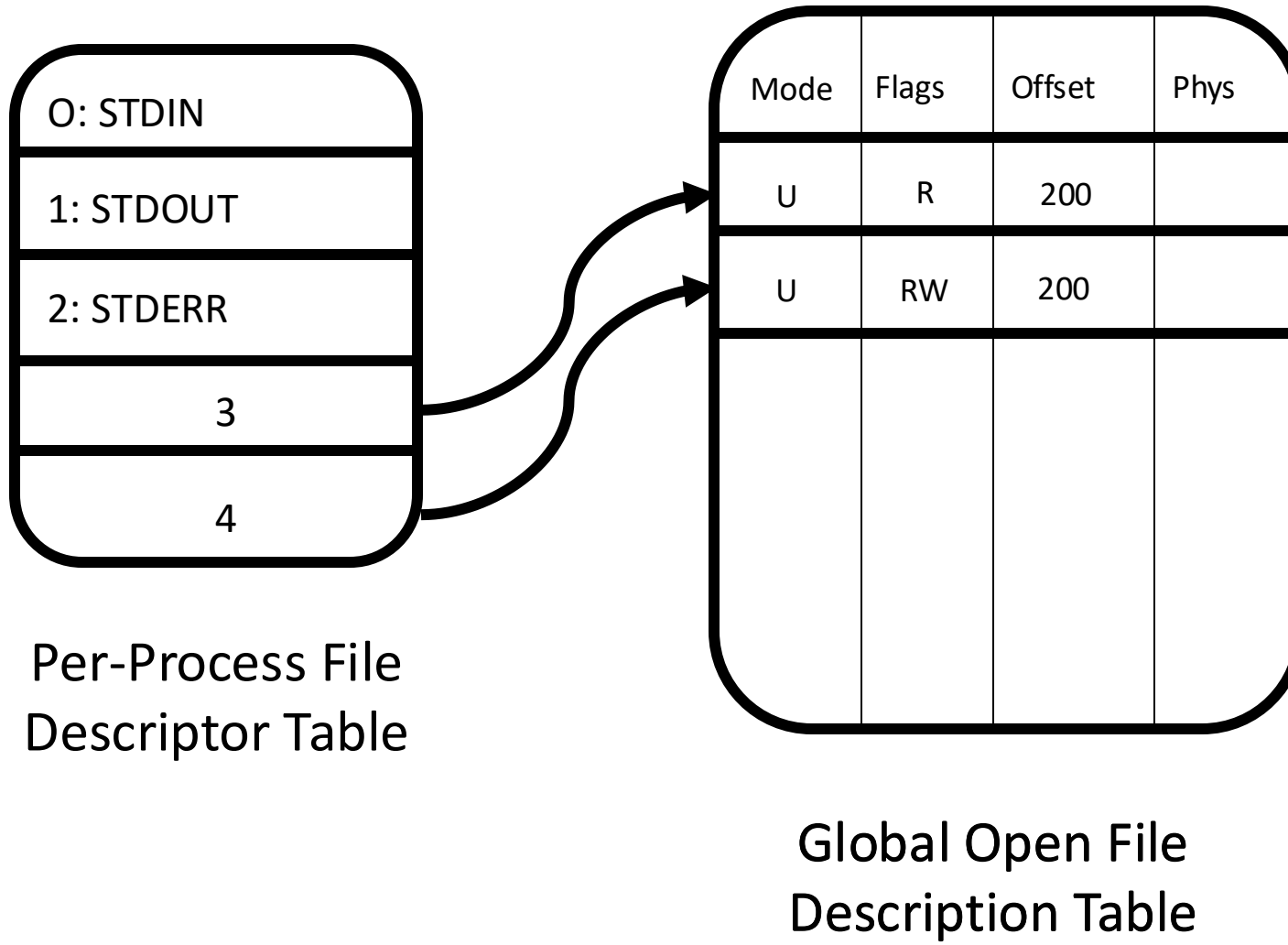
OS Abstraction for Storage: File Systems



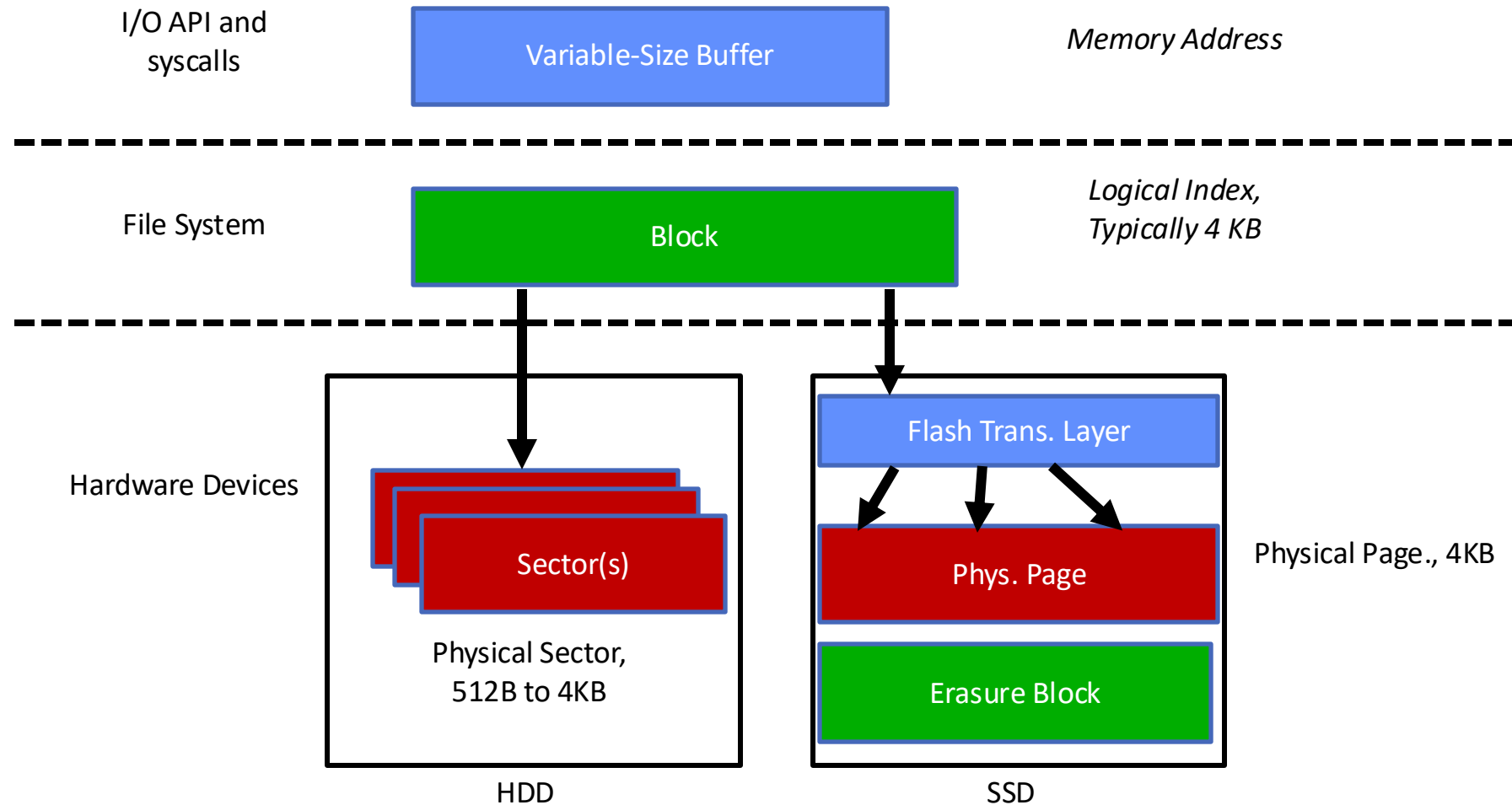
Recall: I/O and Storage Layers



Recall: FD & File Descriptors



From Storage to File Systems



What is a File System?

OS layer that transforms block interface of storage devices into Files, Directories, etc

Purpose of a File System

Take limited hardware interface (array of blocks) and provide a more convenient/useful interface with:

Naming: Find file by name, not block numbers

Organization: Organize file names within directories

Placement: Map files to blocks

Protection: Enforce access restrictions

Reliability: Keep files intact despite crashes, failures, etc.

User vs. System View of a File

User's view:

Durable data structures

System's view (system call interface):

Collection of bytes (UNIX)

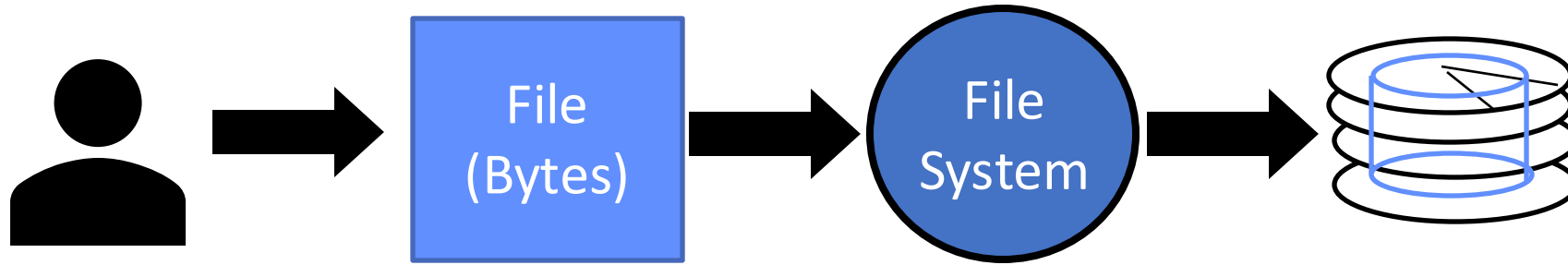
Doesn't matter to system what kind of data structures you want to store on disk!

System's view (inside OS):

Collection of blocks (a block is a logical transfer unit, while a sector is a physical one)

Block size $\geq$ sector size; in UNIX, block size is 4KB

Translation from User to System View



What happens if user says: “give me bytes 2 – 12?”

- Fetch block corresponding to those bytes
- Return just the correct portion of the block

What about writing bytes 2 – 12?

- Fetch block, modify relevant portion, write out block

Everything inside file system is in terms of whole-size blocks

What Does the File System Need to Do?

Track free disk blocks

Need to know where to put newly written data

Track which blocks contain data for which files

Need to know where to read a file from

Track files in a directory

Find list of file's blocks given its name

Where do we maintain all of this?

Somewhere on disk

Critical Factors in File System Design

(Hard) Disk Performance !!!

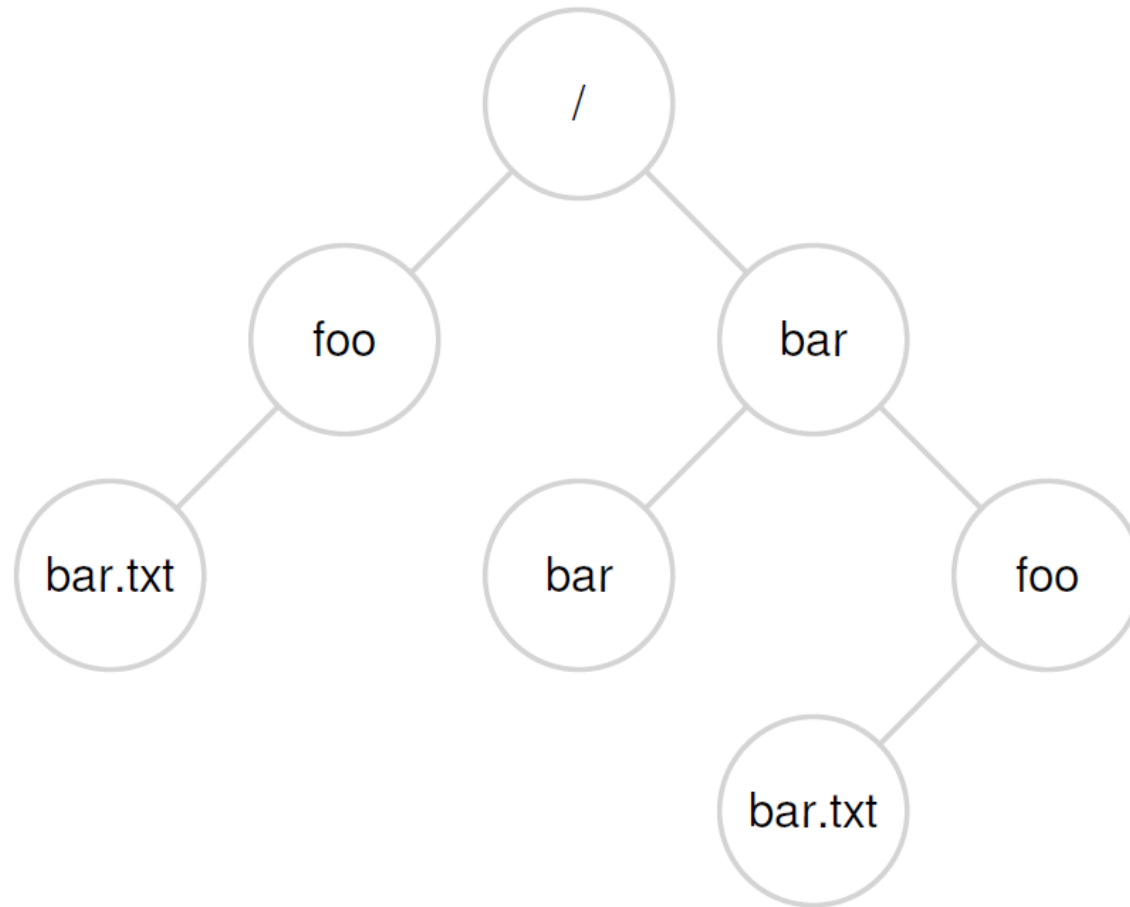
Open before read/write

Size is determined as files are used !!!

Organized into directories

Need to carefully allocate / free blocks

Files & Directories



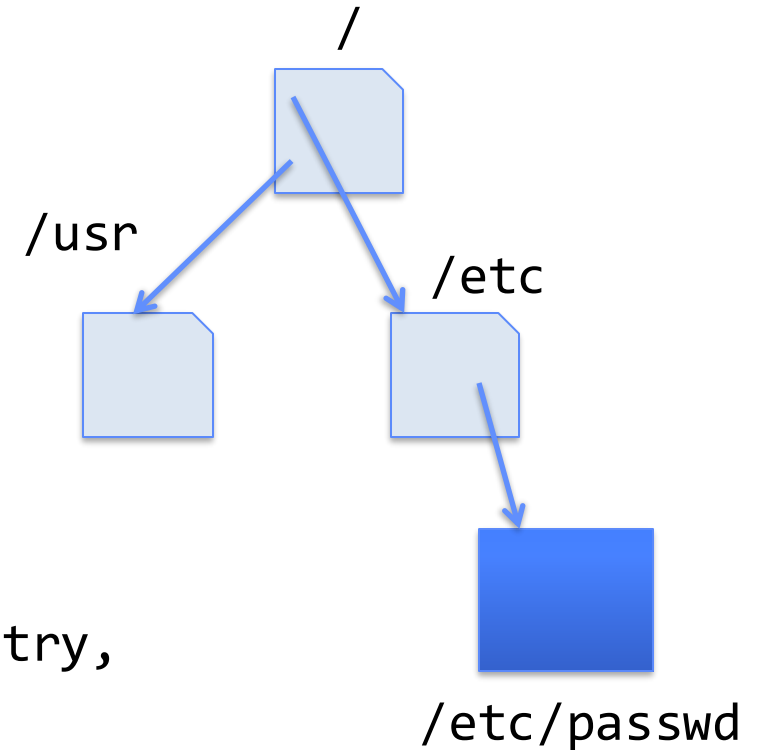
Manipulating Directories

System calls to access directories

- `open` / `creat` / `readdir` traverse the structure
- `mkdir` / `rmdir` add/remove entries
- `link` / `unlink` (rm)

libc support

- `DIR * opendir (const char *dirname)`
- `struct dirent * readdir (DIR *dirstream)`
- `int readdir_r (DIR *dirstream, struct dirent *entry, struct dirent **result)`



Example: Early Unix File System

Superblock object: information about file system

Free bitmaps: what is allocated/not allocated

Inode object: represents a specific file (“index node”)

Dentry object: directory entry, single component of a path

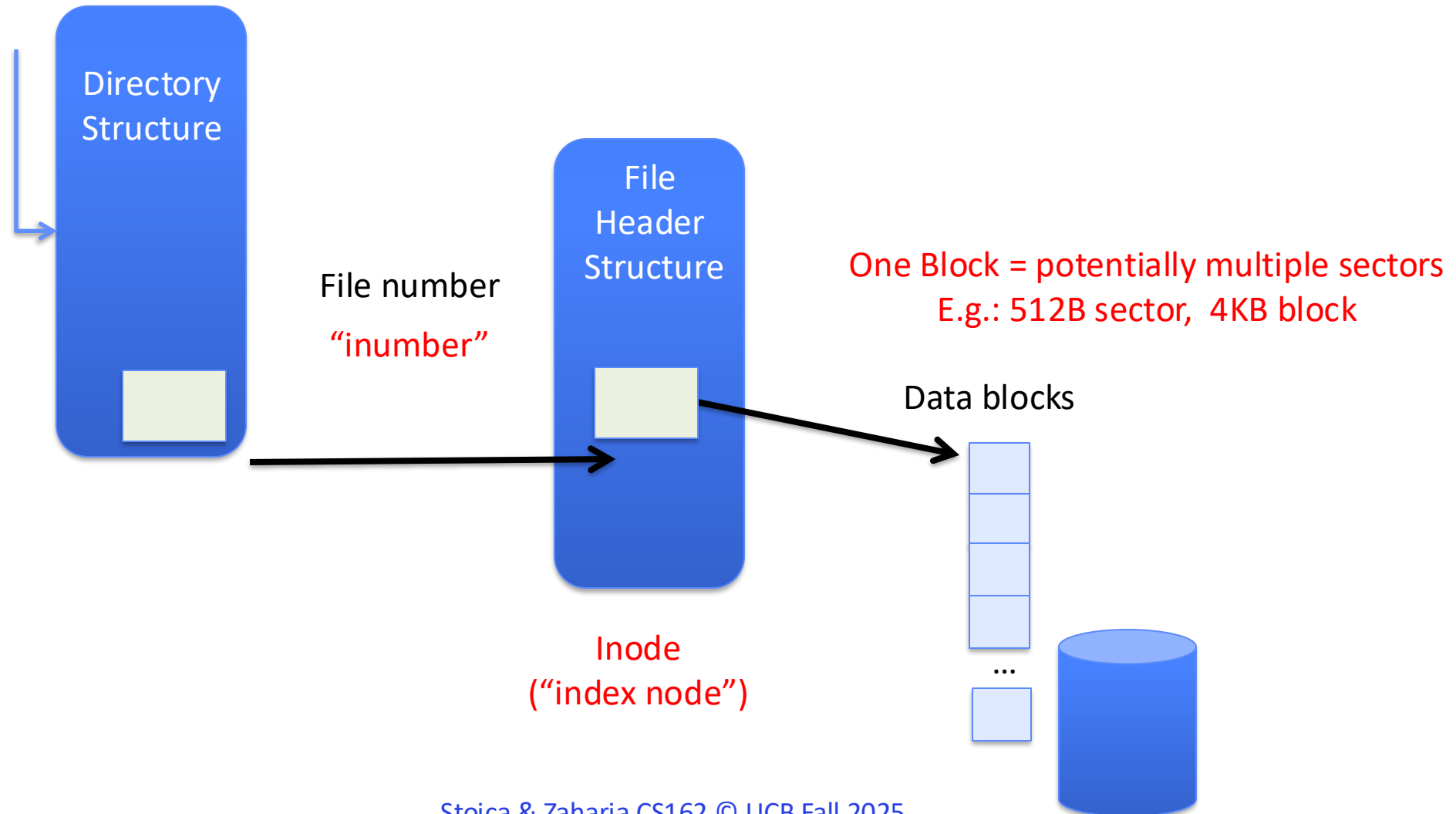
Blocks: How files are stored on disk

File object: open file associated with a process

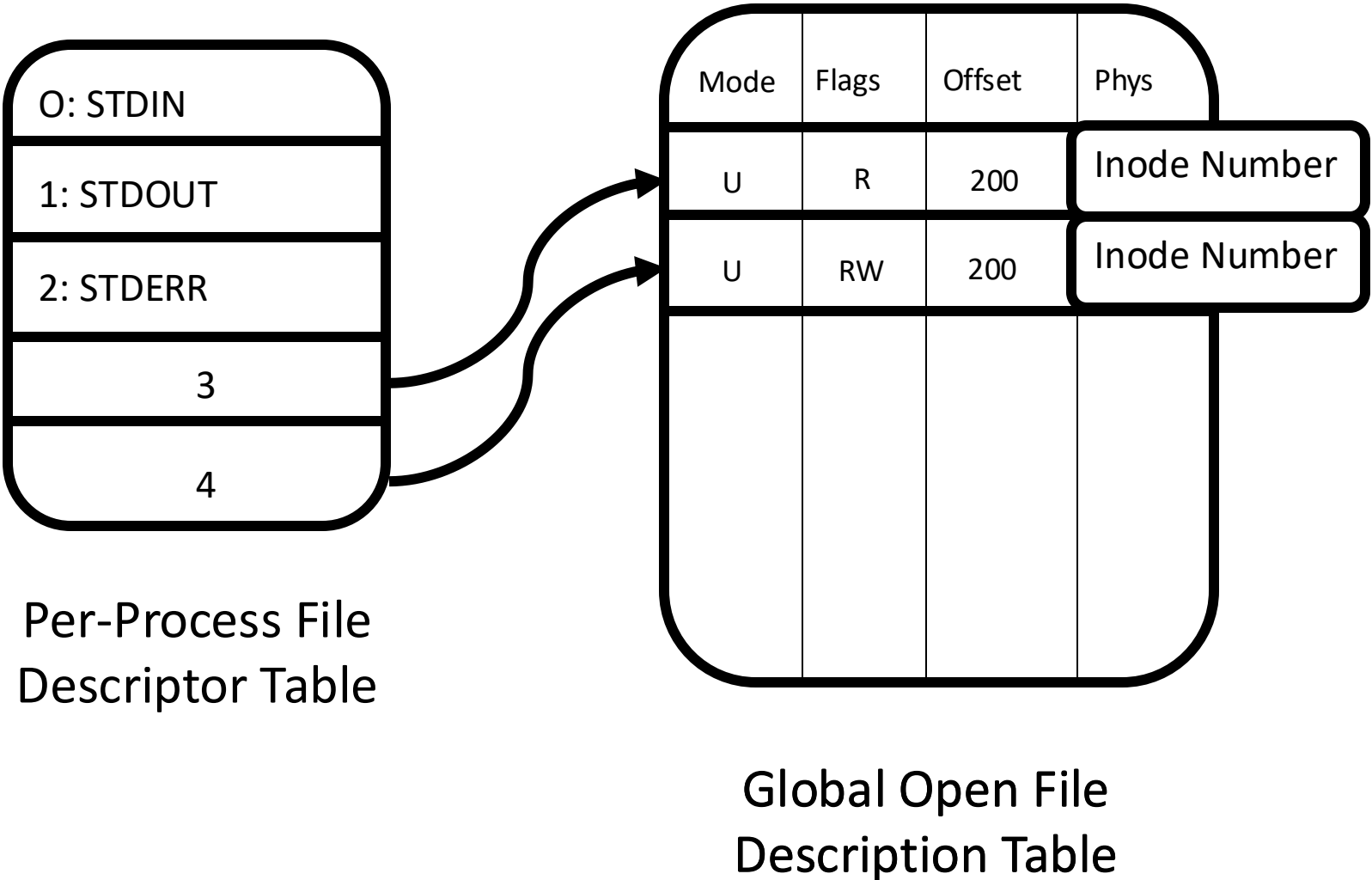
Components of Unix File System

```
open("/home/matei/cs162/foo.txt")
```

File path



The (In)famous Inode



How to Find a File's Inode Number?

Look up through directory hierarchy

A directory is a specialized file containing
<file_name : inode number> mappings

Each <file_name : inode> mapping is called a **directory entry**

How to Read a File from Disk

Let's read file /foo/bar.txt (time goes downwards)

	data	inode	root	foo	bar	root	foo	bar	bar	bar
	bitmap	bitmap	inode	inode	inode	data	data	data	data	data
								[0]	[1]	[2]

How to Read a File from Disk

Let's read file /foo/bar.txt (time goes downwards)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
open(bar)			read	read	read	read	read			
read()					read			read		
read()					read				read	
read()					read					
read()					read					read

File System Workload Characteristics

A Five-Year Study of File-System Metadata

NITIN AGRAWAL

University of Wisconsin, Madison

and

WILLIAM J. BOLOSKY, JOHN R. DOUCEUR, and JACOB R. LORCH

Microsoft Research

Published at FAST 2007

Observation #1: Most Files Are Small

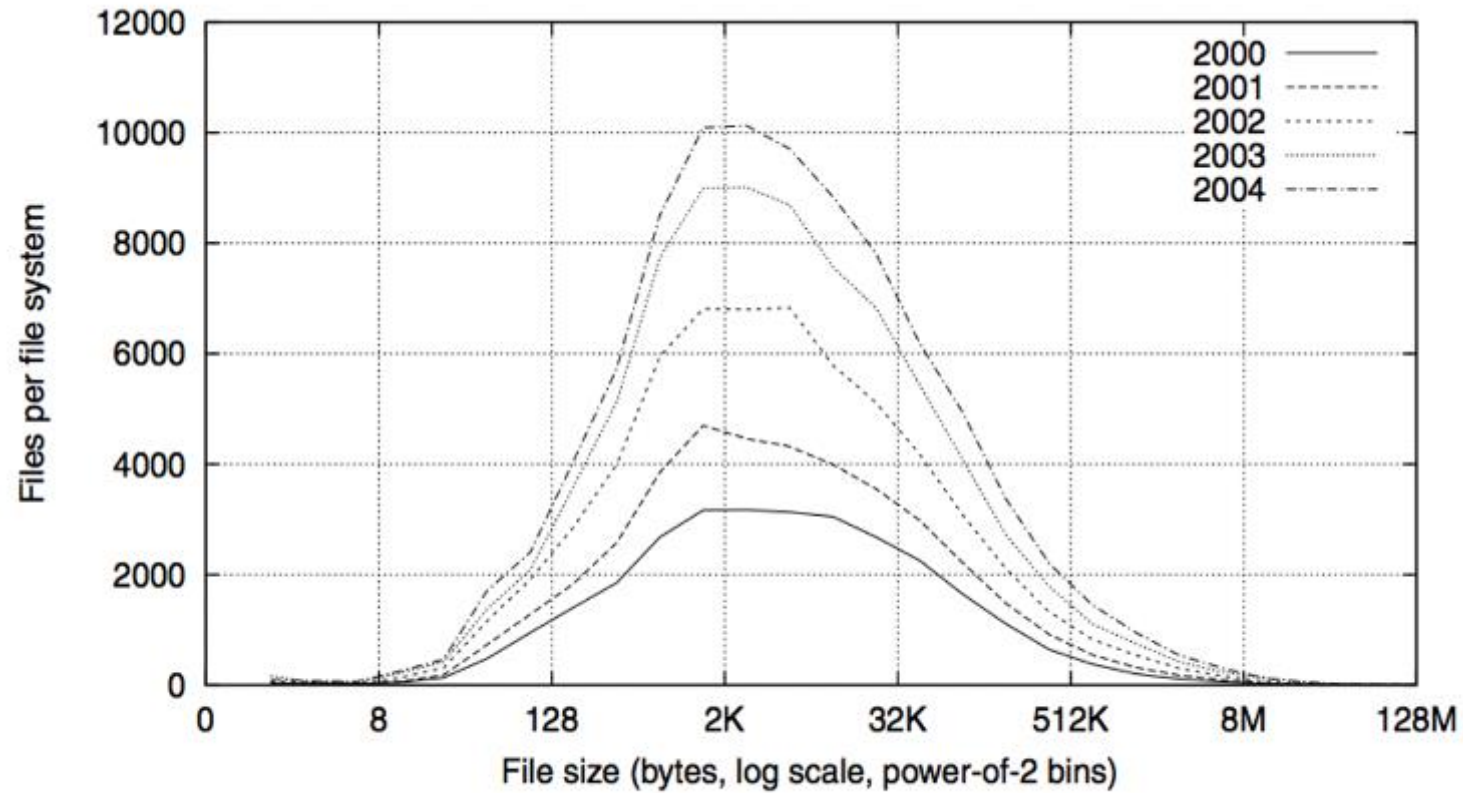


Fig. 2. Histograms of files by size.

Observation #2: Most Bytes are in Large Files

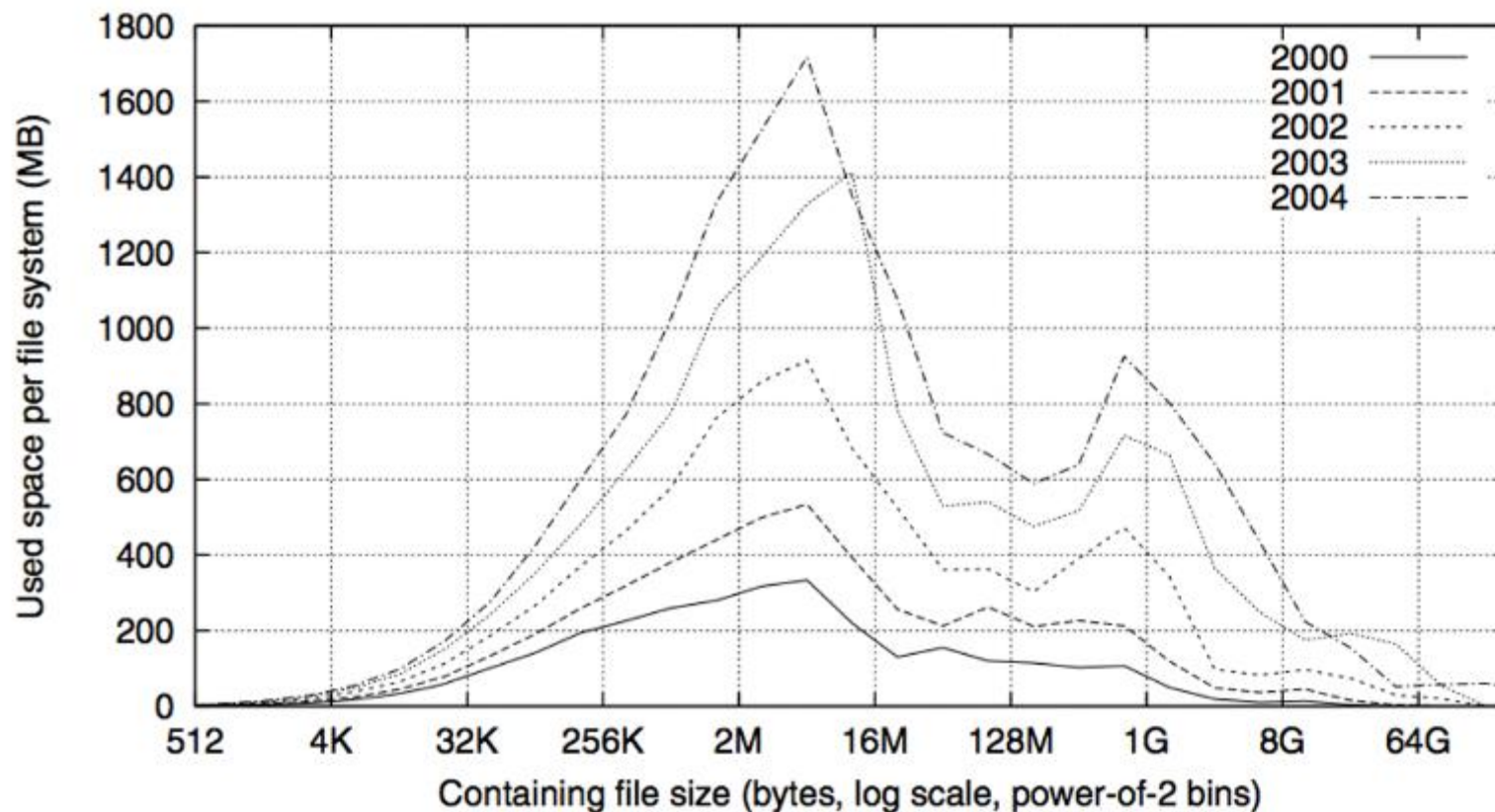


Fig. 4. Histograms of bytes by containing file size.

Unix Inode Structure

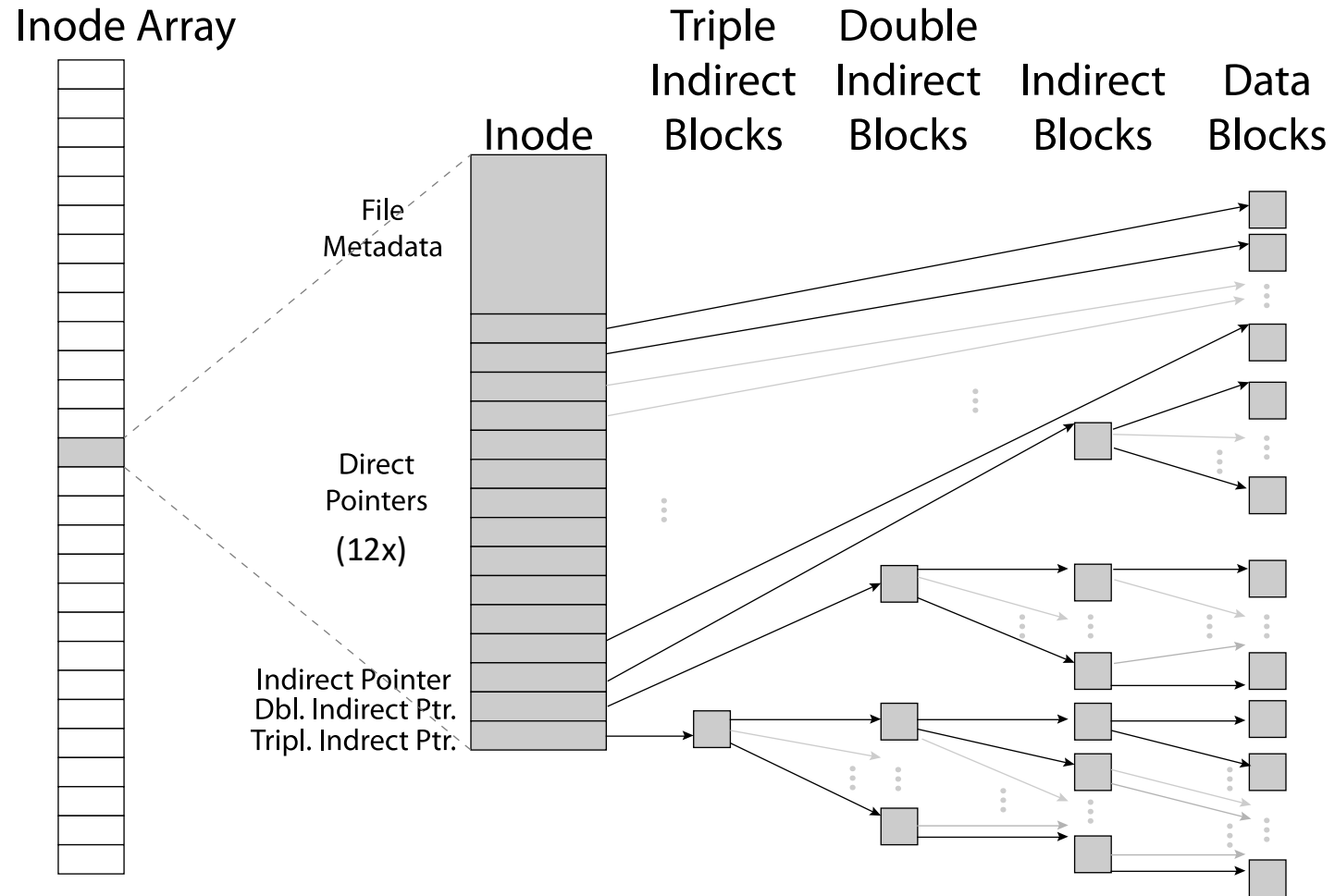
File Number is index into an array of *inode* structures

Each inode corresponds to a file and contains its metadata

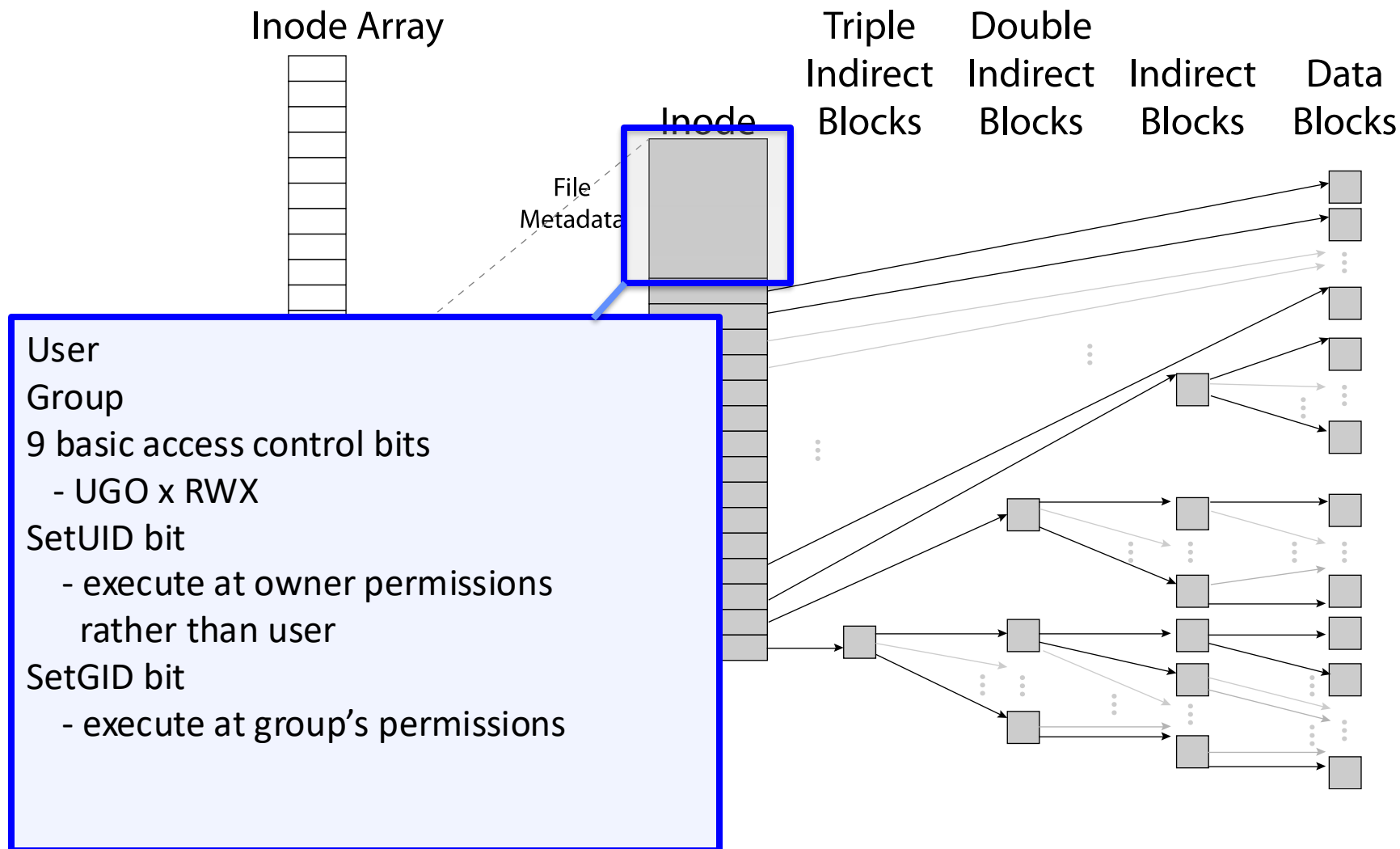
Inode maintains a multi-level tree to find storage blocks for files

Original inode format appeared in BSD 4.1
Berkeley Standard Distribution Unix!

Unix Inode Structure



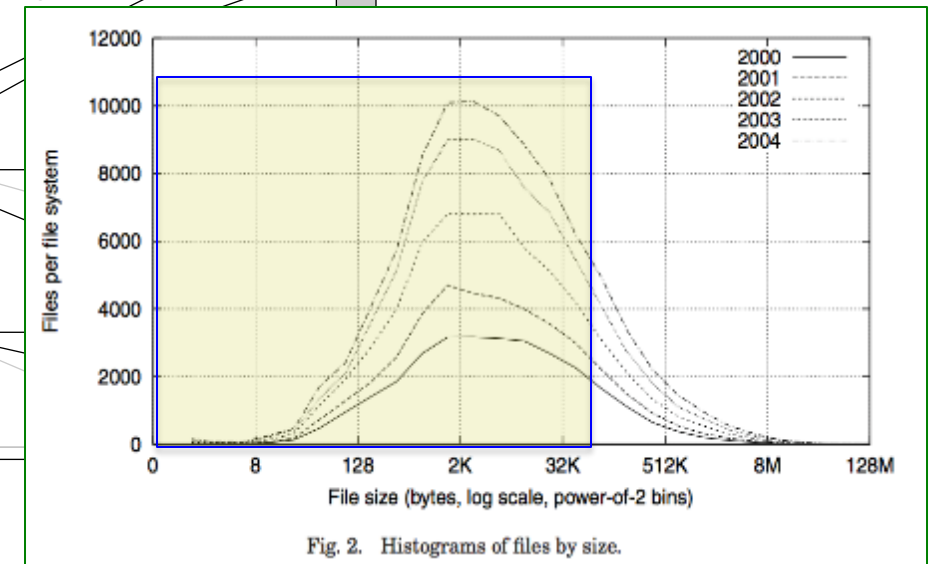
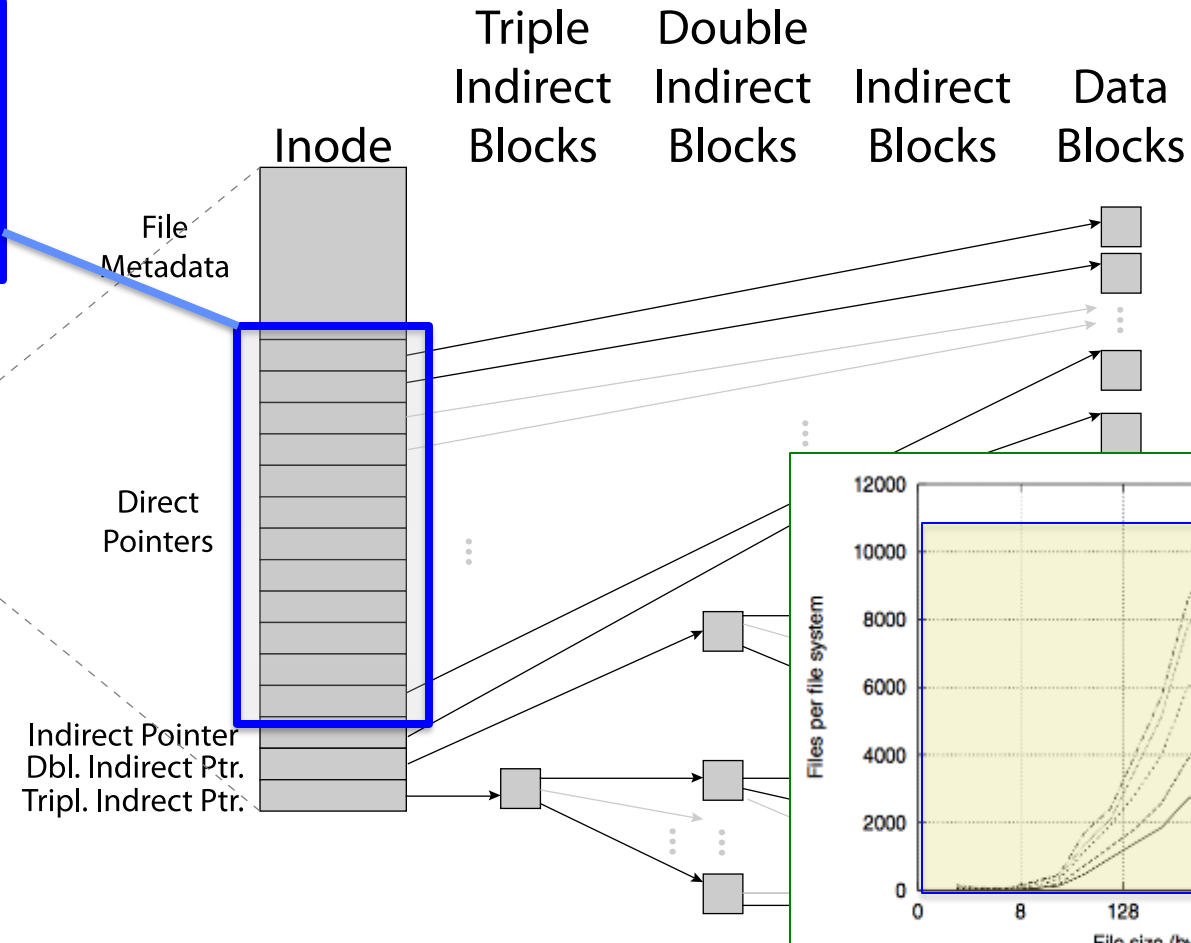
File Attributes



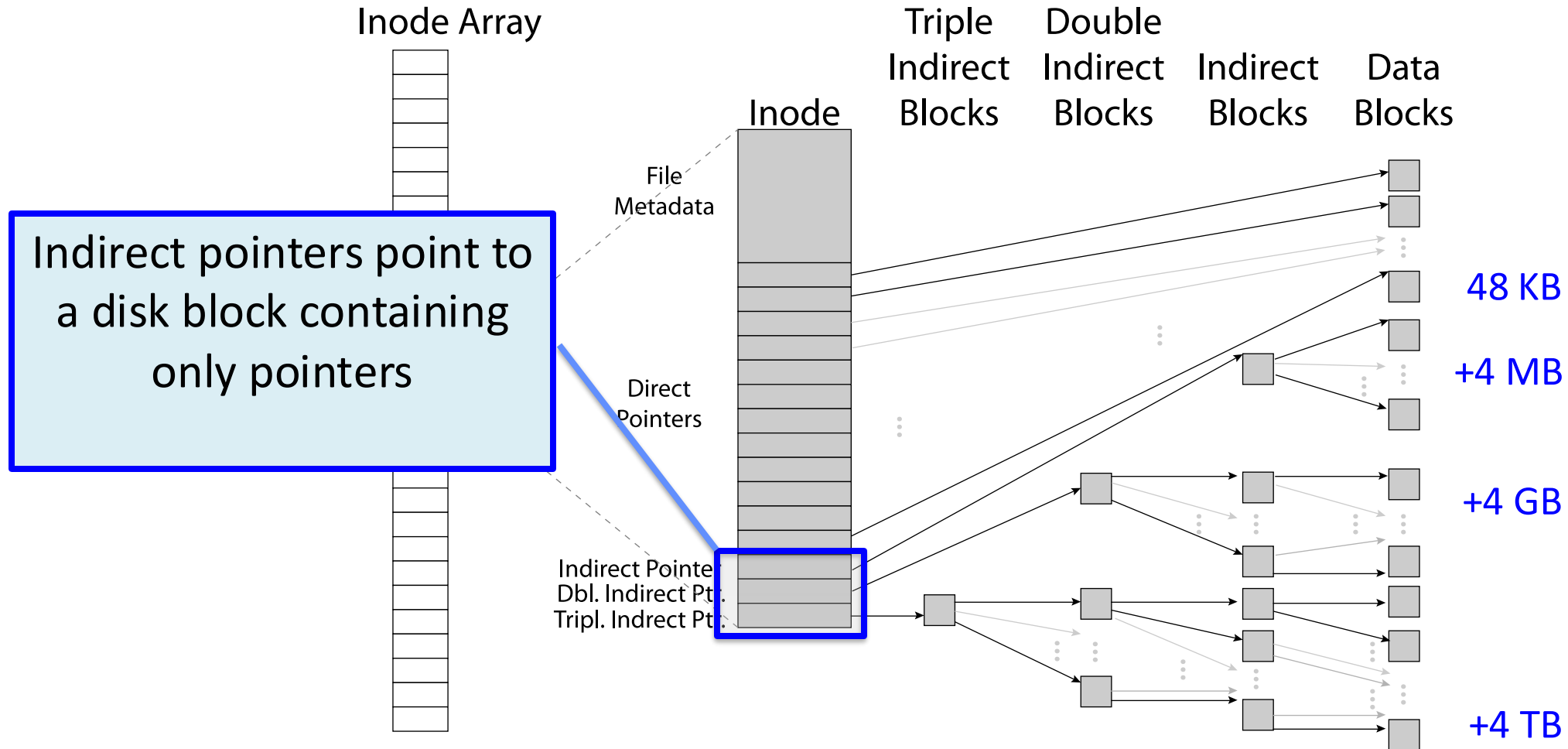
Direct Pointers

12 Direct pointers

4kB blocks $\Rightarrow$ sufficient for files up to 48KB



Indirect Pointers



Indirect Pointers

Assume 4KB blocks

What is the maximum size of a file with only direct pointers?

$$12 * 4 \text{ KB} = 48 \text{ KB}$$

What is the maximum size of a file with one indirect pointer?

$$12 * 4 \text{ KB} + 1024 * 4 \text{ KB} = 4.1 \text{ MB}$$

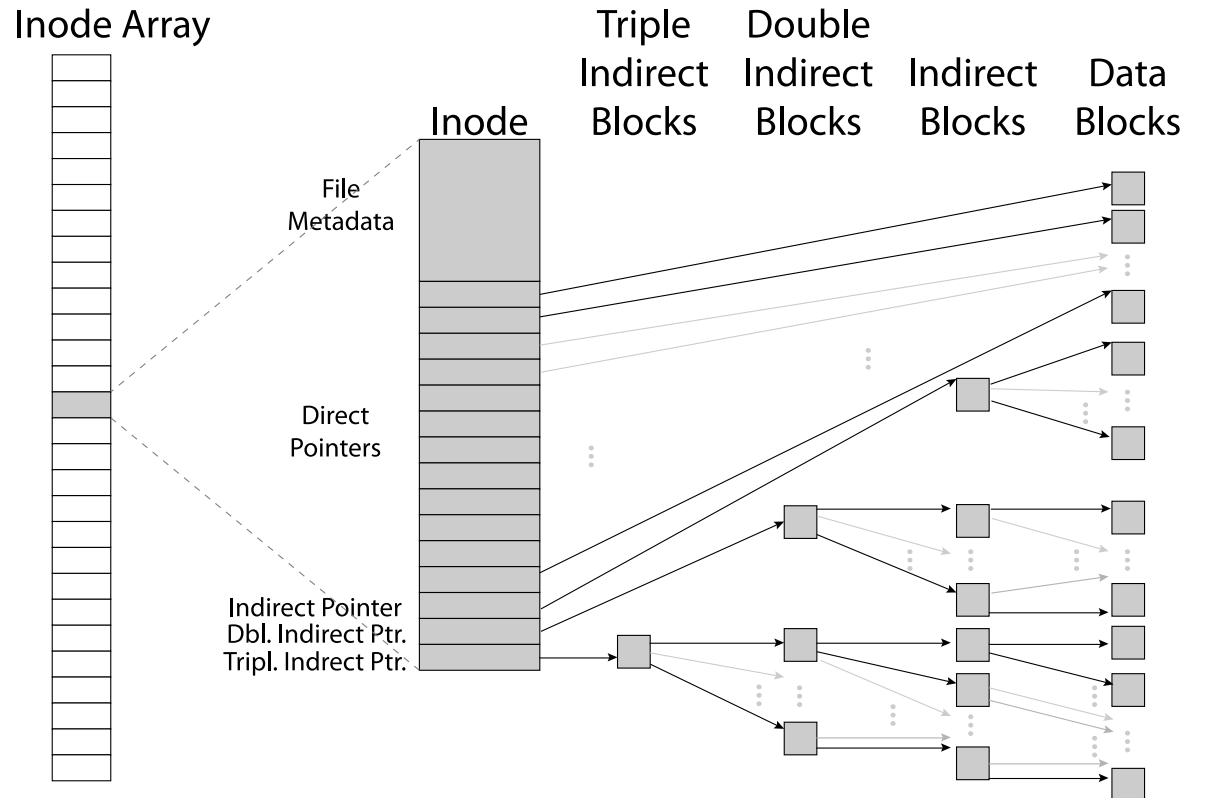
What is the maximum size of a file with double indirect pointers?

$$12 * 4 \text{ KB} + 1024 * 4 \text{ KB} + 1024 * 1024 * 4 \text{ KB} = 4.6 \text{ GB}$$

Inodes form an on-disk index

Sample file in multilevel indexed format:

- 12 direct ptrs, 4K blocks
- How many accesses for block #23? (assume file header accessed on open)?
 - » Two: One for indirect block, one for data
- How about block #5?
 - » One: One for data
- Block #1100?
 - » Three: double indirect block, indirect block, and data



Creating new files

Inodes are (logically) stored in an inode table

File system stores a bitmap of free inodes and free blocks

On creating a new file,

- 1) Check which inode is free/where that inode is stored
- 2) Check which data blocks are free

Putting it together

/cs162/matei.txt (60KB)

Each block is 4KB

Inode is 256 Bytes

Putting it together

/cs162/matei.txt (60KB)



Sblock

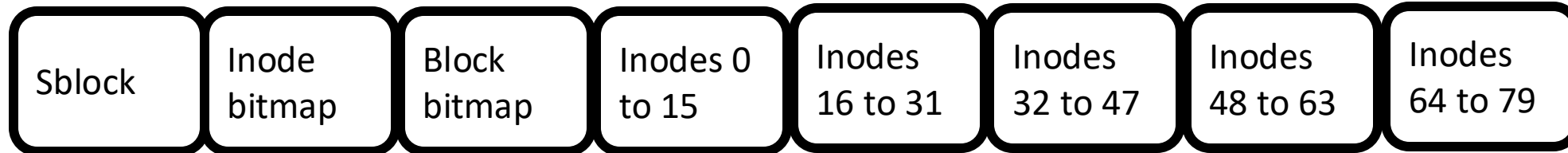
Putting it together

/cs162/matei.txt (60KB)



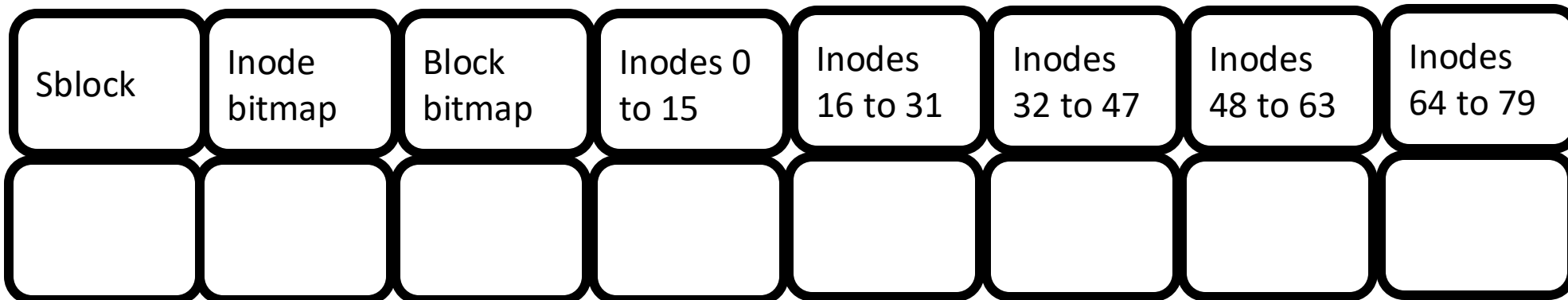
Putting it together

/cs162/matei.txt (60KB)



Putting it together

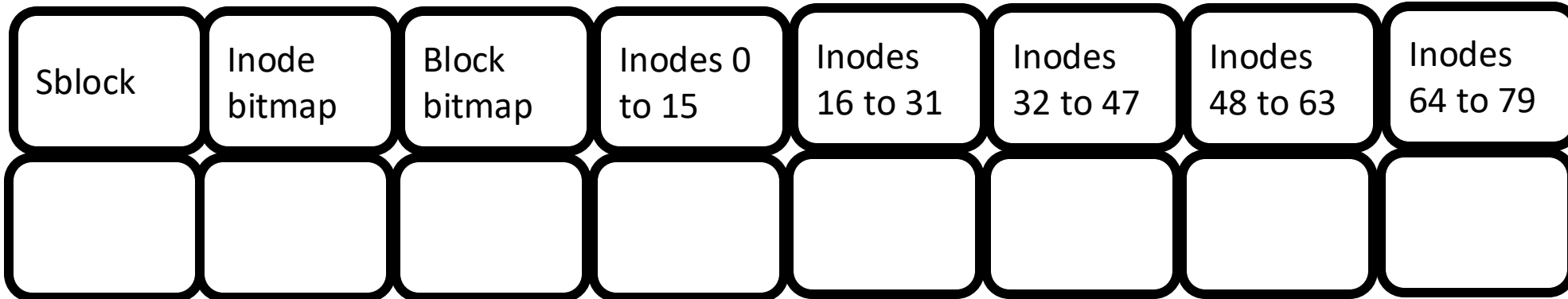
/cs162/matei.txt (60KB)



Putting it together

/

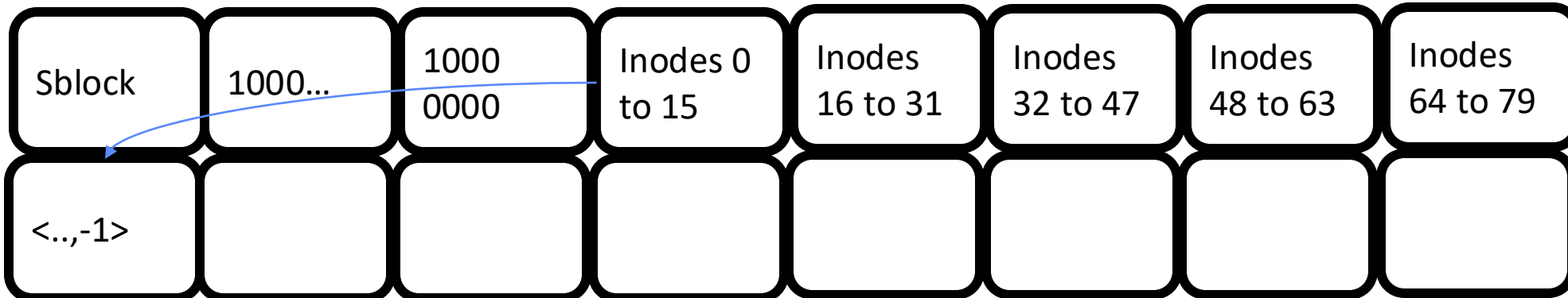
Allocate inode 0
Create data block



Putting it together

/

Allocate inode 0
Create data block



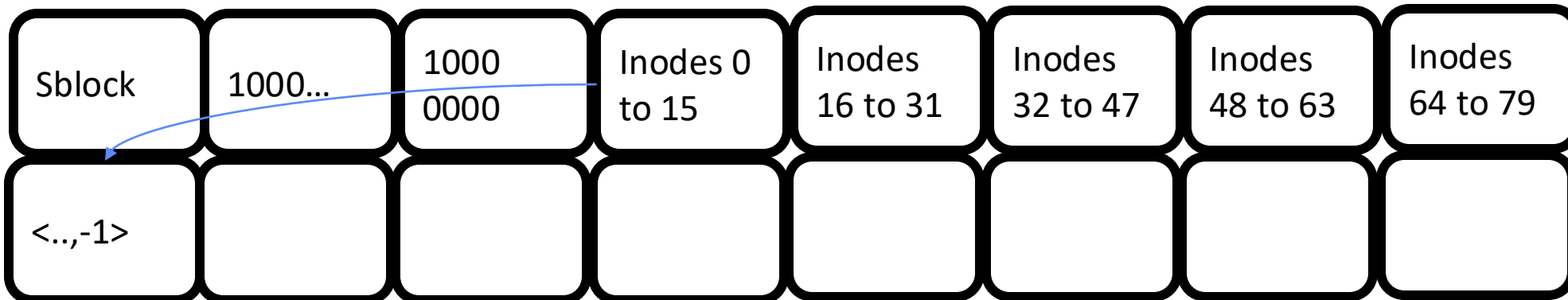
Putting it together

/cs162

Allocate inode 1

Update direntry for /

Create data block



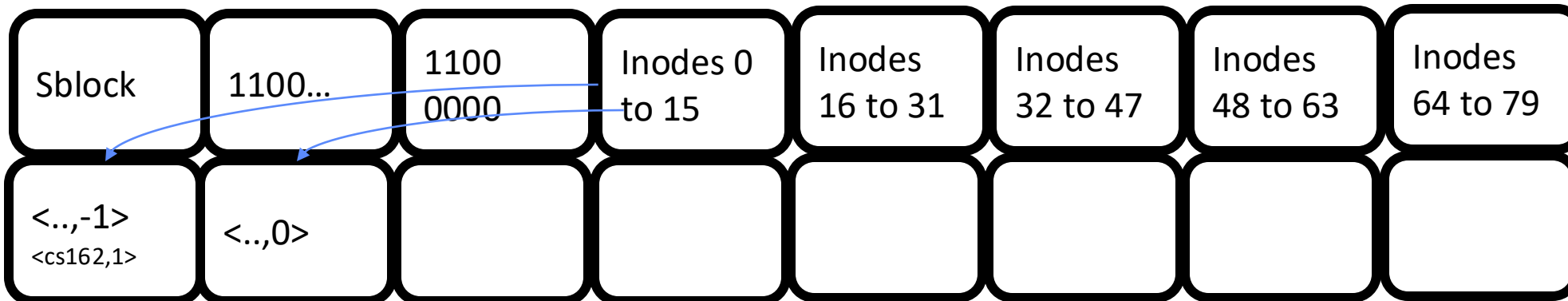
Putting it together

/cs162

Allocate inode 1

Update direntry for /

Create data block



Putting it together

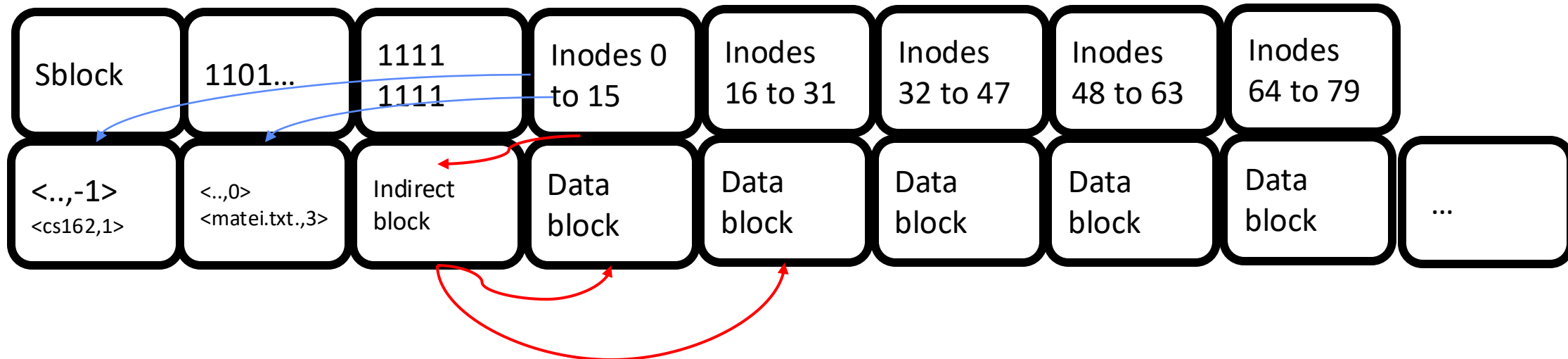
/cs162/matei.txt (60KB)

Allocate inode 3

Update dentry

Create indirect block

Create data blocks



Improving the Unix FS: Berkeley Fast File System

A Fast File System for UNIX*

*Marshall Kirk McKusick, William N. Joy†,
Samuel J. Leffler‡, Robert S. Fabry*

Computer Systems Research Group
Computer Science Division
Department of Electrical Engineering and Computer Science
University of California, Berkeley
Berkeley, CA 94720

ABSTRACT

A reimplementation of the UNIX file system is described. The reimplementation provides substantially higher throughput rates by using more flexible allocation policies that allow better locality of reference and can be adapted to a wide range of peripheral and processor characteristics. The new file system clusters data that is sequentially accessed and provides two block sizes to allow fast access to large files while not wasting large amounts of space for small files. File access rates of up to ten times faster than the traditional UNIX file system are experienced. Long needed enhancements to the pro-

Added disk-aware optimizations

Recall: Critical Factors in File System Design

(Hard) Disk Performance !!!

Maximize sequential access, minimize seeks

Open before Read/Write

- Can perform protection checks and look up where data is in advance

Size is determined as files are used !!!

- Can write (or read zeros) to expand the file
- Start small and grow, need to make room

Organized into directories

- What data structure (on disk) for that?

Need to carefully allocate / free blocks

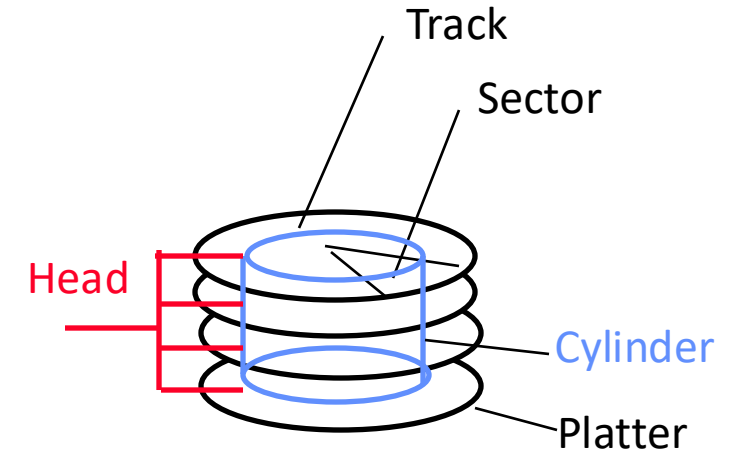
- Such that access remains efficient

Recall: Magnetic Disks

Cylinders: all the tracks under the head at a given point on all surfaces

Read/write data is a three-stage process:

- **Seek time:** position the head/arm over the proper track
- **Rotational latency:** wait for desired sector to rotate under r/w head
- **Transfer time:** transfer a block of bits (sector) under r/w head



Fast File System (BSD 4.2, 1984)

Same inode structure as in BSD 4.1

- Same file header and triply indirect blocks like we just studied
- Some changes to block sizes from 1024 $\Rightarrow$ 4096 bytes for performance

Optimization for Performance and Reliability:

- Distribute inodes among different tracks to be closer to data
- Uses bitmap allocation in place of freelist
- Attempt to allocate files contiguously
- 10% reserved disk space
- Skip-sector positioning (mentioned later)

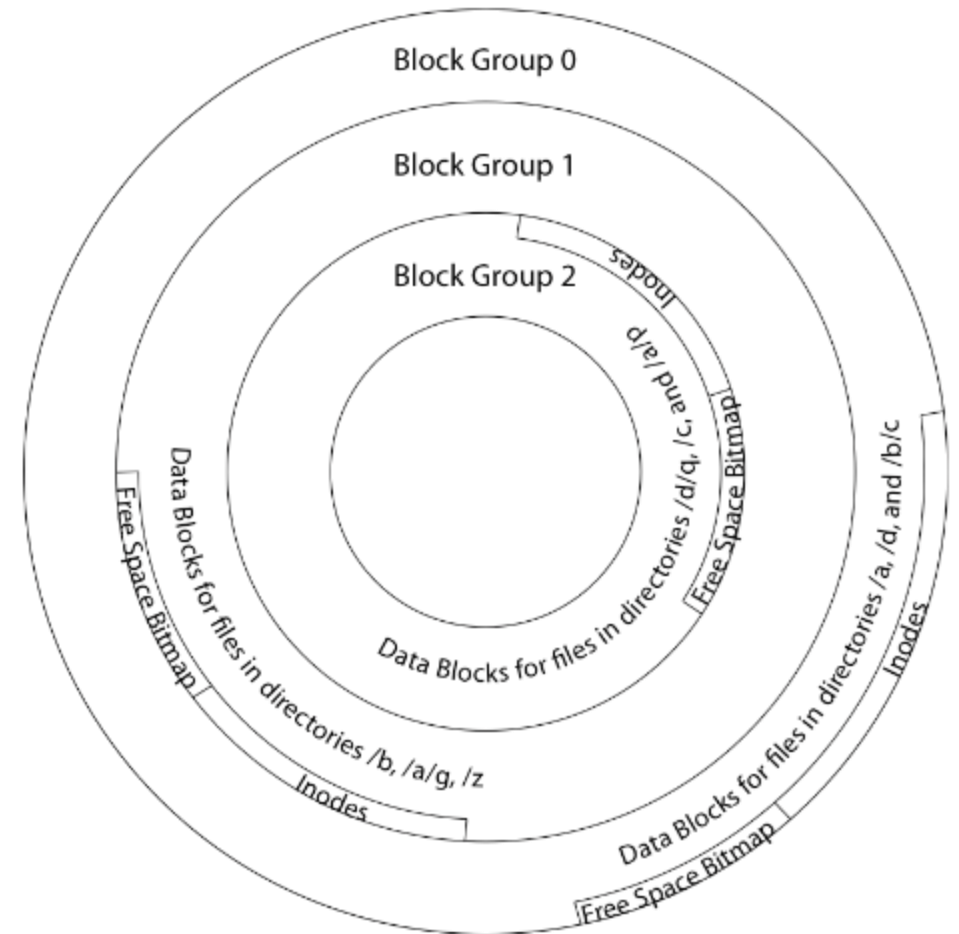
FFS Locality: Block Groups

Distribute header information (inodes) closer to the data blocks, in same “cylinder group”

File system volume divided into set of “block groups”

Data blocks, metadata, and free space interleaved within block group

Put a directory and its files in same block group



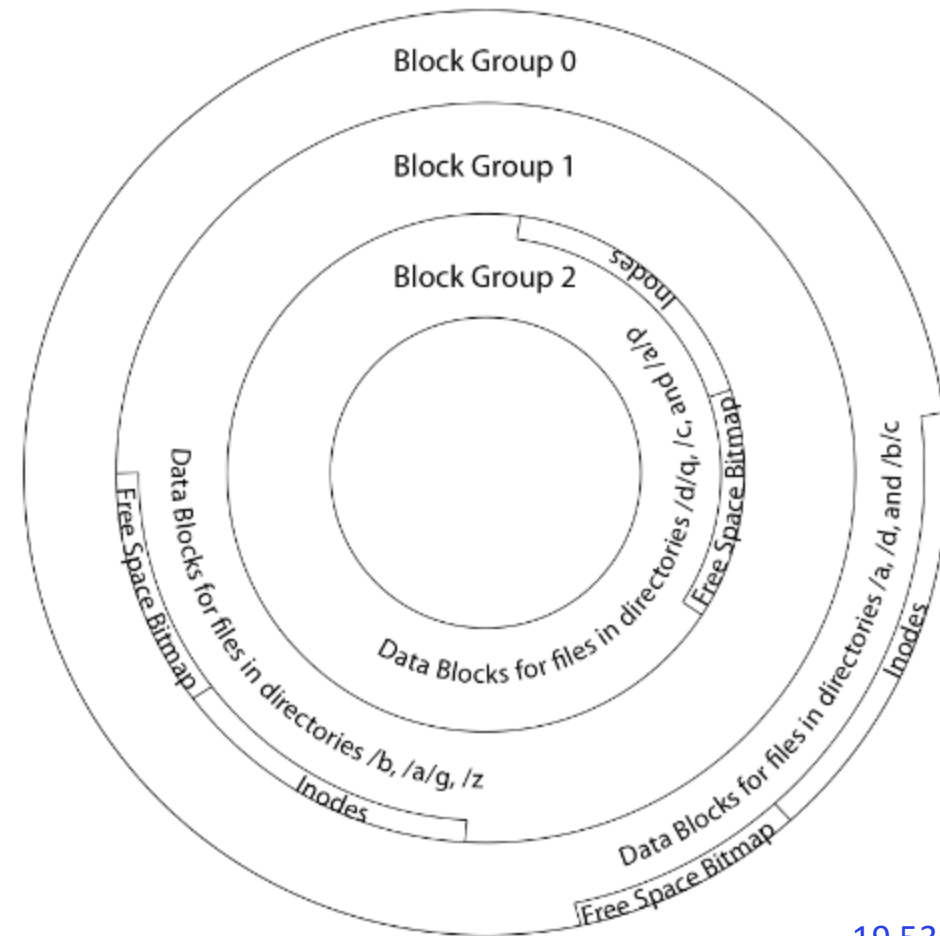
FFS Locality: Block Groups

First-Free allocation of new file blocks

- To expand file, first try successive blocks in bitmap, then choose a new range of blocks
- Few little holes at start, big sequential runs at end of group
- Avoids fragmentation
- Sequential layout for big files

Important: keep 10% or more free!

- Reserve space in the Block Group



Attack of the Rotational Delay

Missing blocks due to rotational delay

Issue: Read one block, do processing, and read next block. In meantime, disk has continued turning: missed next block! Need 1 revolution/block!

Attack of the Rotational Delay

Solution 1: Skip sector positioning (“interleaving”)

- » Place the blocks from one file on every other block of a track: give time for processing to overlap rotation
- » Can be done by OS or in modern drives by the disk controller

Solution 2: Read-ahead: read next block right after first, even if application hasn’t asked for it yet

- » This can be done by the OS
- » Or by the disk (“track buffers”): many modern disk controllers have internal RAM that allows them to read a complete track

UNIX 4.2 BSD FFS

Pros

- Efficient storage for both small and large files
- Locality for both small and large files
- Locality for metadata and data
- No defragmentation necessary!

Cons

- Inefficient for tiny files (a 1 byte file requires both an inode and a data block)
- Inefficient encoding when file is mostly contiguous on disk
- Need to reserve 10-20% of free space to prevent fragmentation

What about other file systems?

**FAT:
File Allocation Table
(MS-DOS,1977)**

Windows NTFS

FAT (File Allocation Table)

Assume (for now) we have a way to translate a path to a “file number”

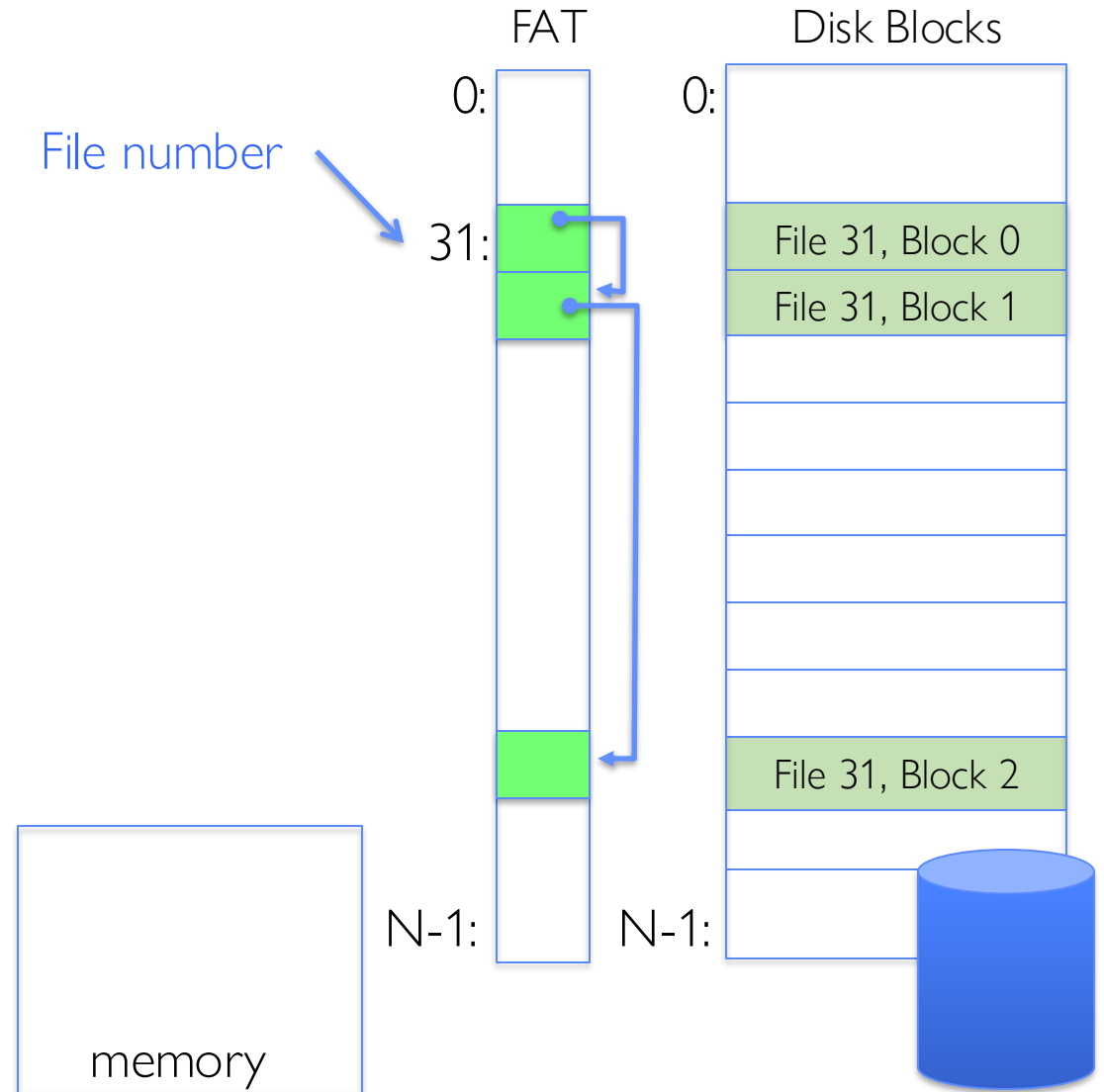
- i.e., a directory structure

Disk Storage is a collection of Blocks

- Just hold file data
(offset $o = \langle B, x \rangle$)

Example: `file_read 31, < 2, x >`

- Index into FAT with file number
- Follow linked list to block
- Read the block from disk into memory

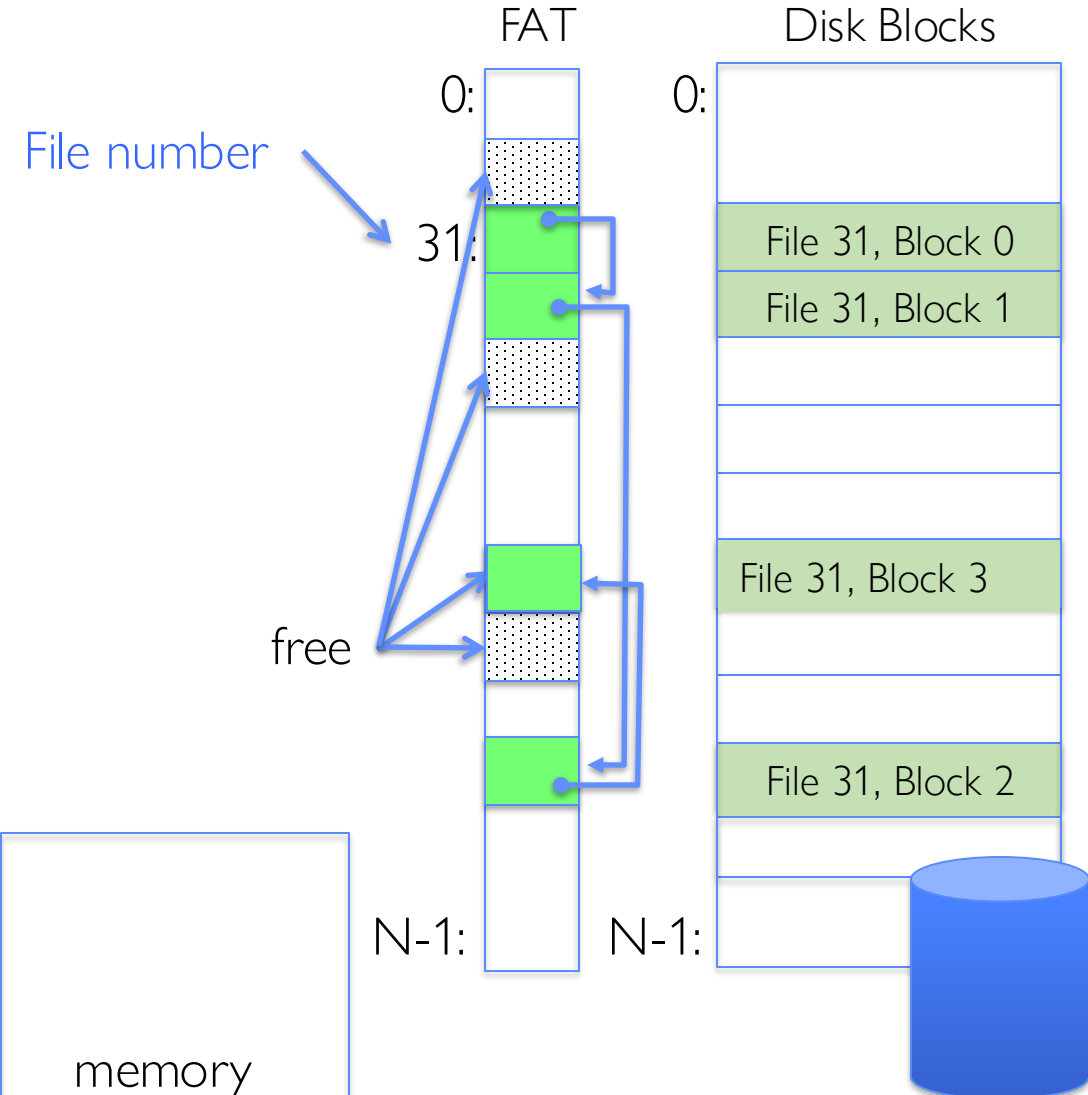


- Unused blocks marked free
 - Could require scan to find
 - Or, could use a free list



```
file_write(31, < 3, y >)
```

- Grab free block
- Linking them into file



Where is FAT stored?

- On disk

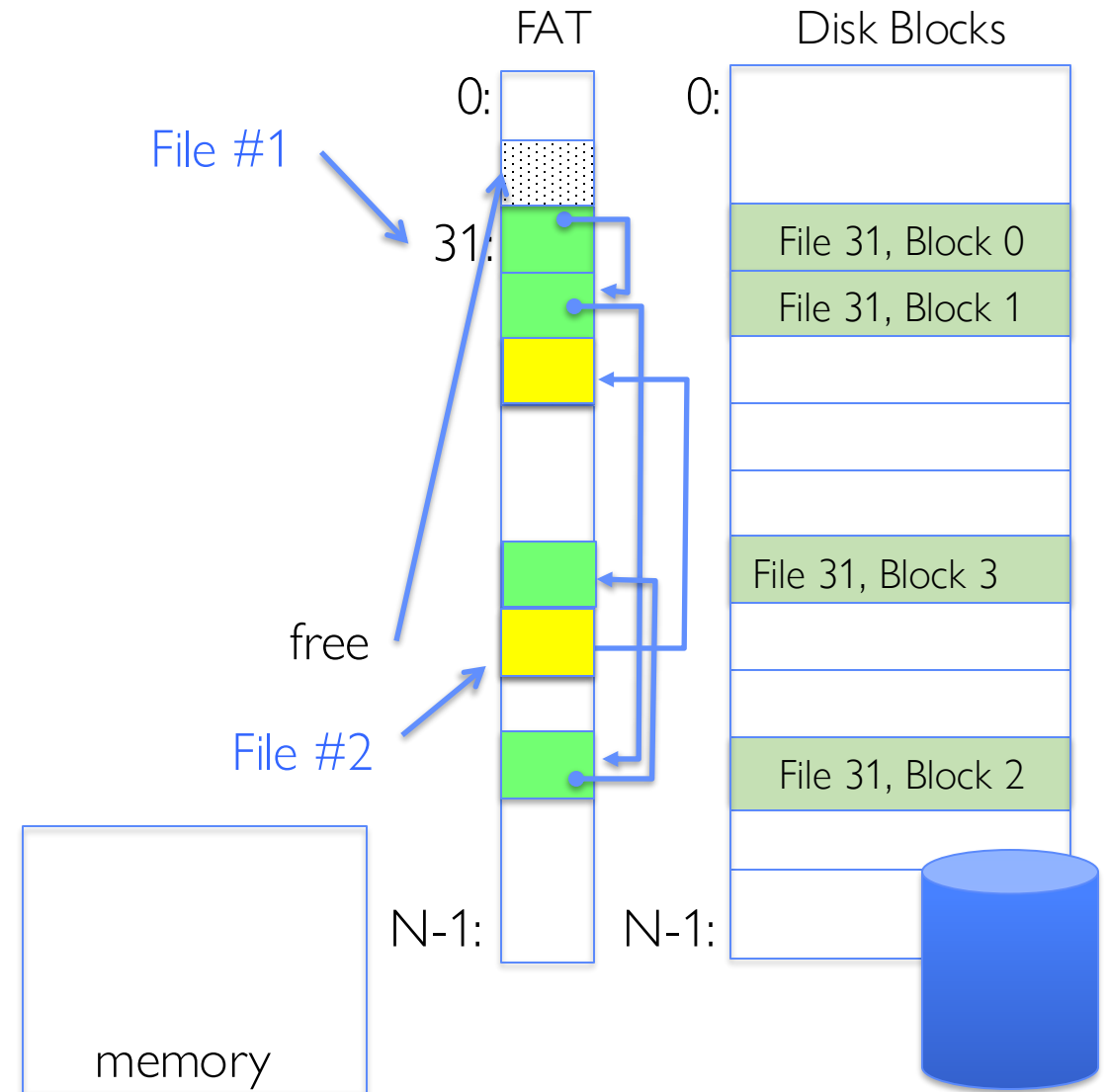
How to format a disk?

- Zero the blocks, mark FAT entries “free”

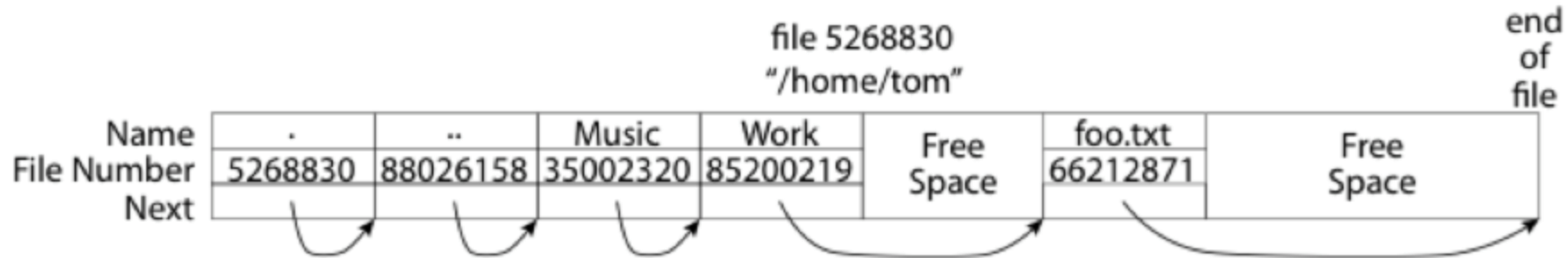
How to quick format a disk?

- Mark FAT entries “free”

Simple: can implement in device firmware



FAT: Directories



A directory is a file containing <file_name: file_number> mappings

In FAT: file attributes are kept in directory (!!!)

- Not directly associated with the file itself

Each directory a linked list of entries

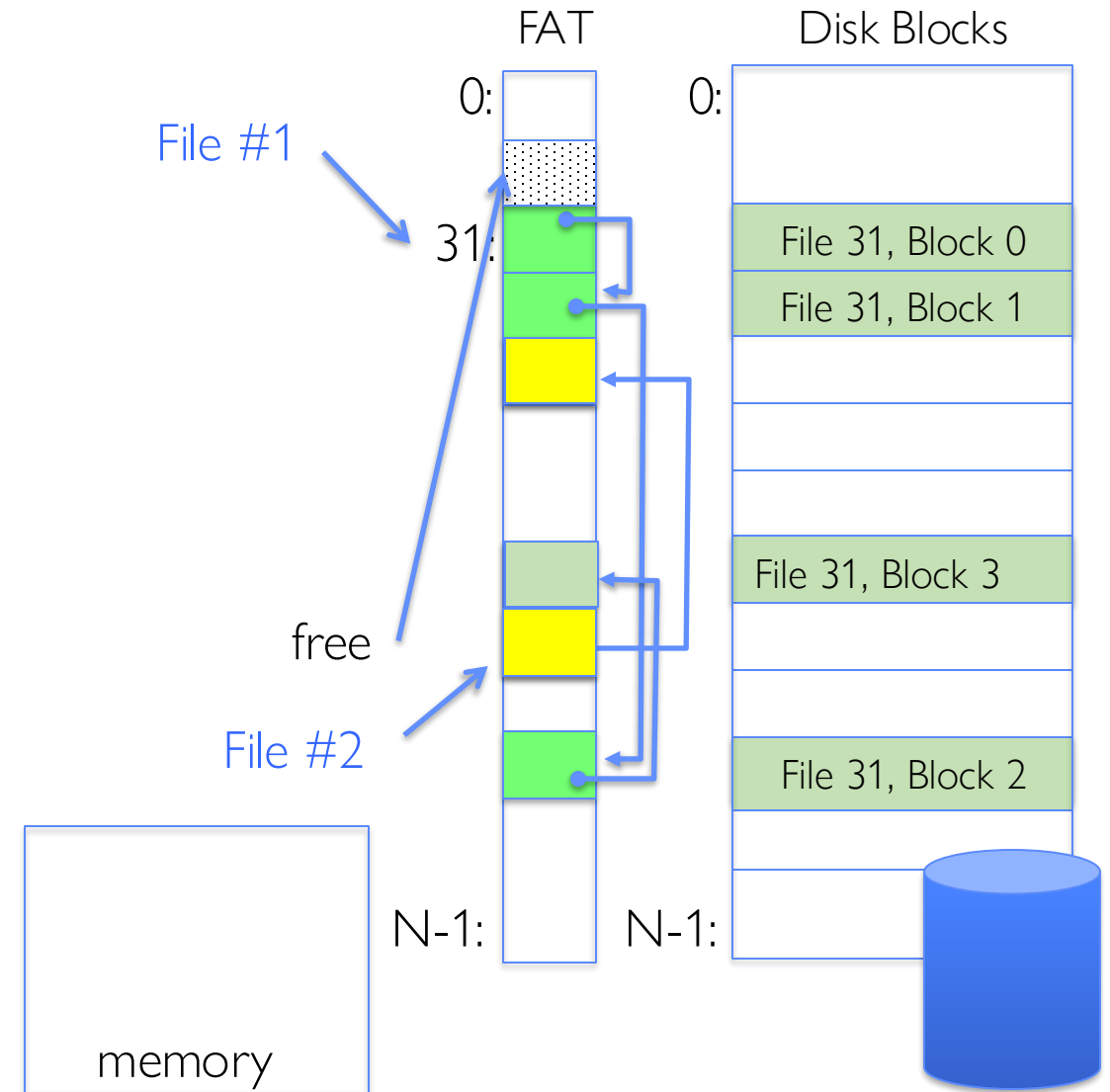
- Requires linear search of directory to find particular entry

Where do you find root directory ("/")?

- At well-defined place on disk
- For FAT, this is at block 2 (there are no blocks 0 or 1)

Suppose you start with the file number:

- Time to find first block?
- Block layout for file?
- Sequential access?
- Random access?
- Fragmentation?
- Small files?
- Big files?



Windows NTFS

New Technology File System (NTFS)

Default on modern Windows systems

Variable length extents

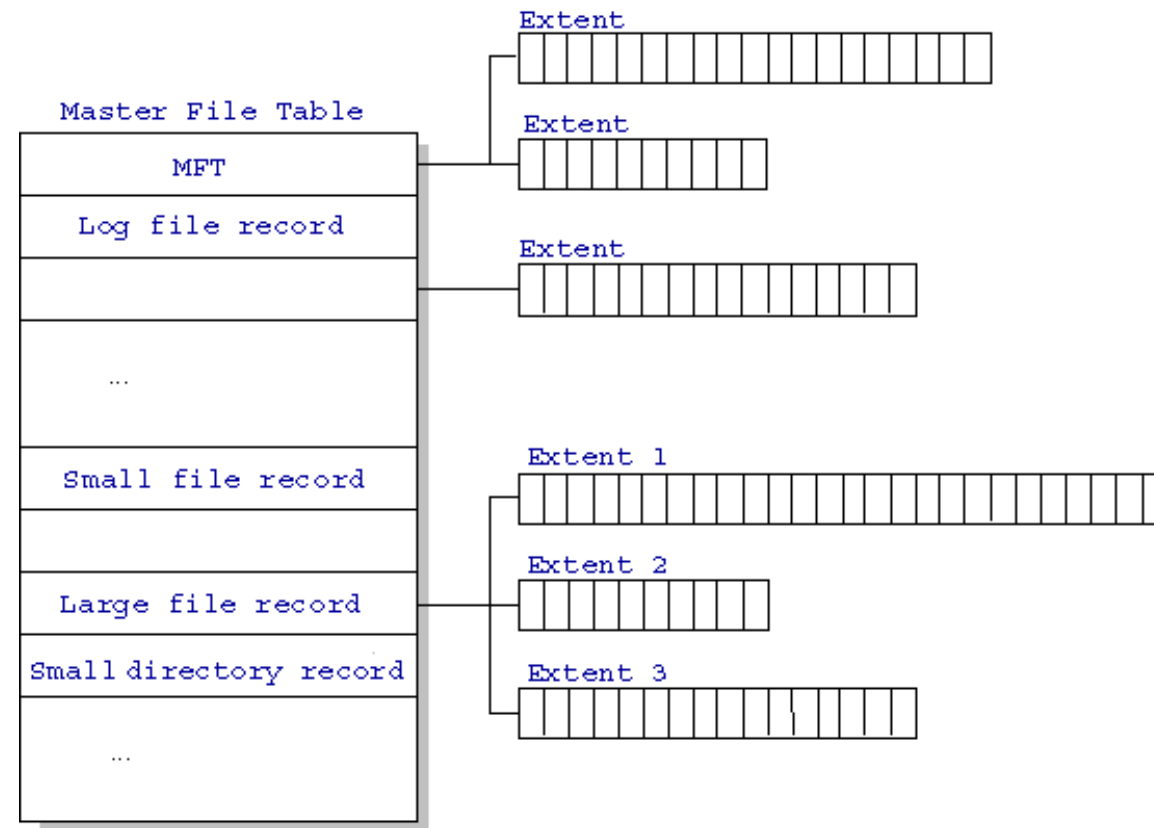
Master File Table

- Everything (almost) is a sequence of <attribute:value>

Each entry in MFT contains metadata and:

- File's data directly (for small files)
- A list of *extents* (start block, size) for file's data
- For big files: pointers to other MFT entries with *more* extent lists

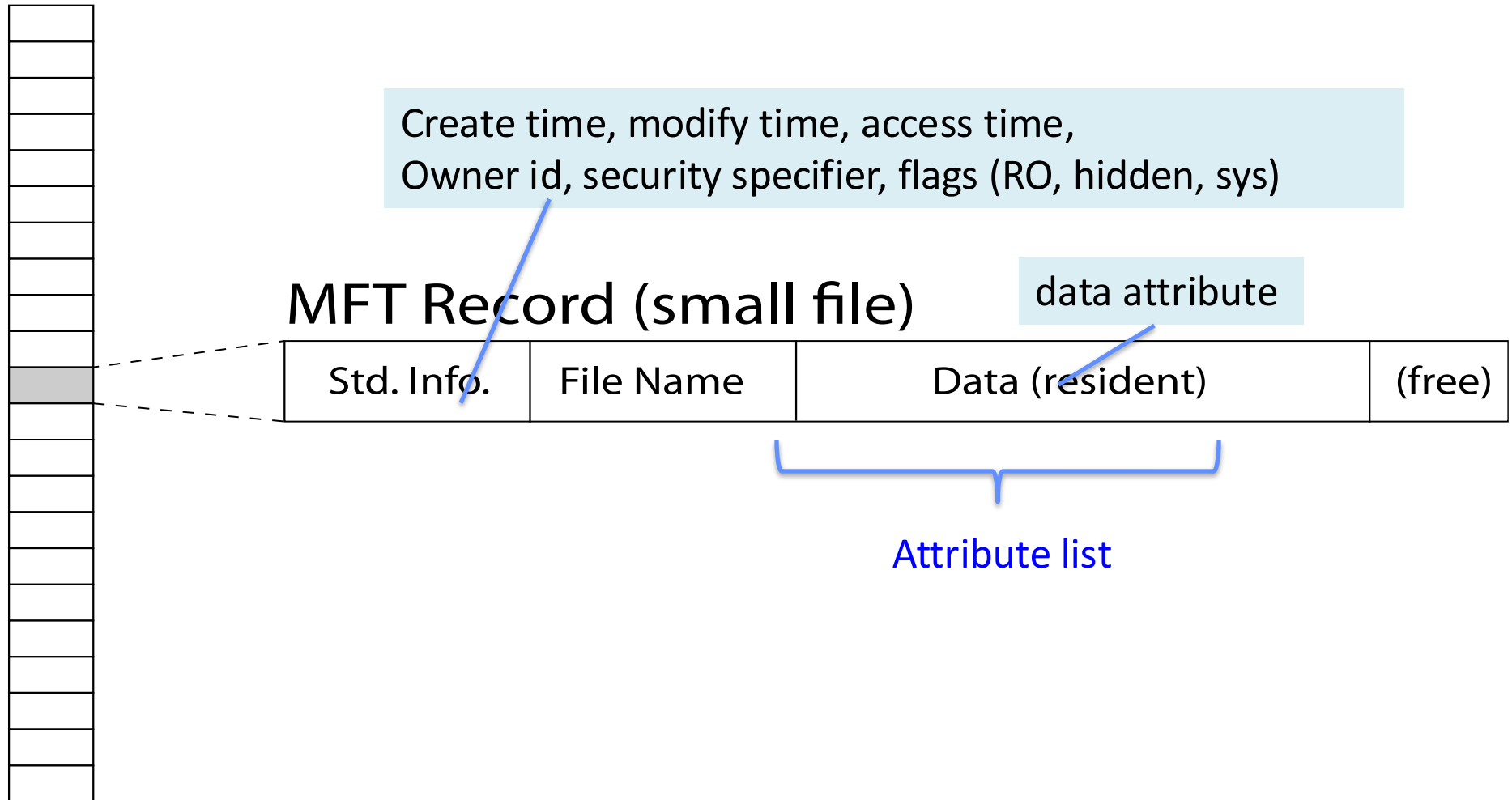
NTFS



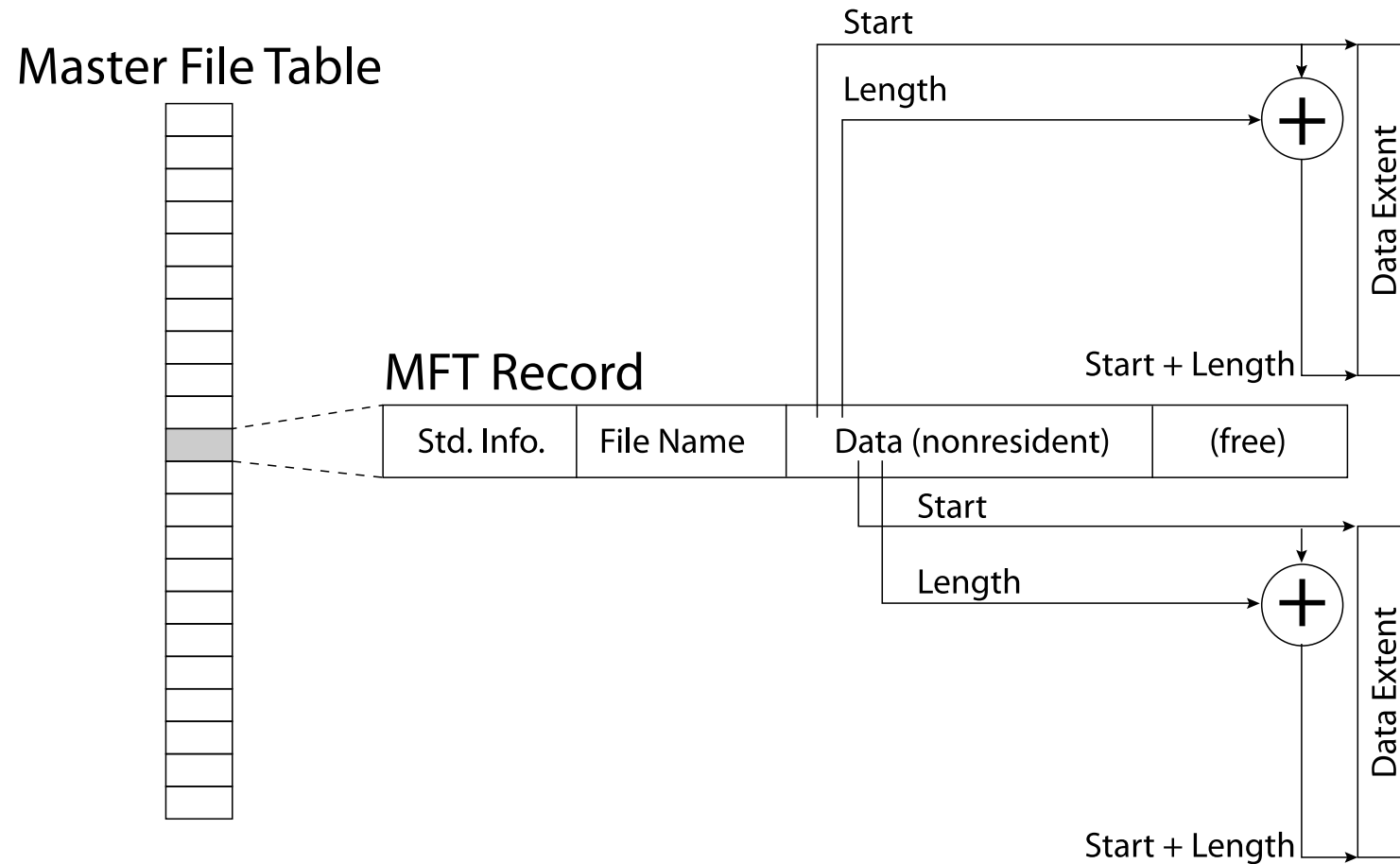
<http://ntfs.com/ntfs-mft.htm>

NTFS Small File: Data stored with Metadata

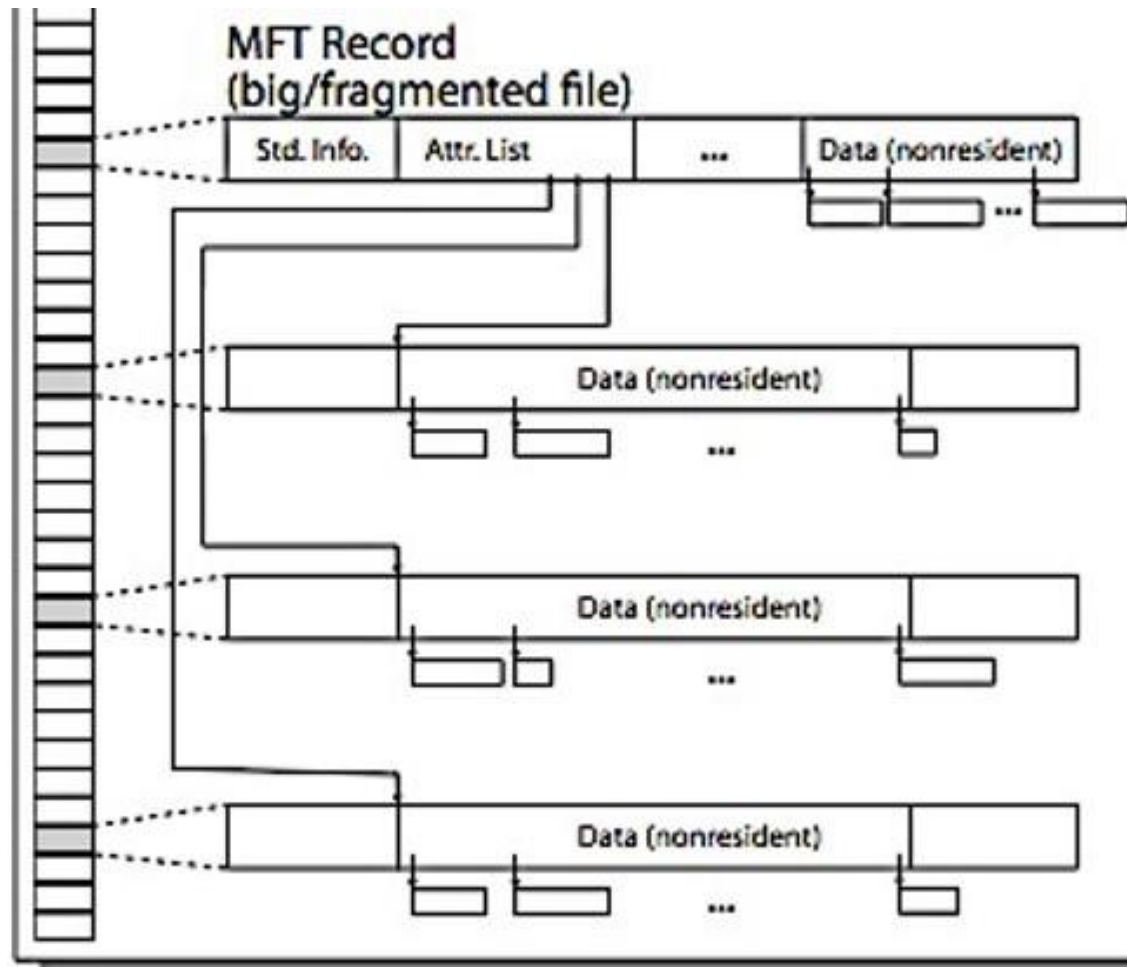
Master File Table



NTFS Medium File: Extents for File Data



NTFS Large File: Pointers to Other MFT



NTFS Directories

Directories implemented as B-Trees

File's number identifies its entry in MFT

MFT entry always has a file name attribute

- Human readable name, file number of parent dir

Hard link? Multiple file name attributes in MFT entry